

UPSCALE PRO

An AI-Powered Serverless Image Upscaling Platform

DISSERTATION / MAJOR PROJECT REPORT

Submitted by:

AMIR SHAMIM

Enrollment No: 2023-301-021

In partial fulfilment for the award of the degree of
BACHELOR OF COMPUTER APPLICATION (BCA)

Under the supervision of:

Dr. Sapna Jain



Department of Computer Science & Engineering,

School of Engineering Sciences & Technology

JAMIA HAMDARD

New Delhi – 110062

(2026)

DECLARATION

I, Mr. Amir Shamim, a student of Bachelor of Computer Application (BCA), Enrollment No: 2023-301-021, hereby declare that the dissertation entitled “Upscale Pro: An AI-Powered Serverless Image Upscaling Platform” which is being submitted by me to the Department of Computer Science, School of Engineering Sciences & Technology, Jamia Hamdard, New Delhi, in partial fulfilment of the requirement for the award of the degree of Bachelor of Computer Application, is my original work and has not been submitted anywhere else for the award of any Degree, Diploma, Associateship, Fellowship, or other similar title or recognition.

Date: April, 2026

Place: New Delhi

Amir Shamim

ACKNOWLEDGEMENT

It is a pleasure to acknowledge the many individuals who knowingly and unknowingly helped me to complete this project. First and foremost, I thank the Almighty for all the blessings that carried me through these years of study.

I extend my deepest gratitude to Dr. Sapna Jain Ma'am, my project supervisor, who has consistently supported, guided, and encouraged me to explore advanced cloud computing paradigms and artificial intelligence techniques. Her expertise in evaluating system architectures and her patient mentorship at every stage of development proved invaluable. The insights she provided regarding scalable microservice patterns and serverless deployment strategies were instrumental in shaping the final architecture of Upscale Pro.

I also extend my sincere thanks to the faculty members of the Department of Computer Science & Technology at Jamia Hamdard for cultivating a rigorous academic environment. Their lectures on database management, software engineering, and network security laid the theoretical foundation upon which this project was built.

I am grateful to the non-teaching staff who ensured uninterrupted access to laboratory resources and internet connectivity throughout the development cycle.

Amir Shamim

Enrollment No: 2023-301-021

ABSTRACT

Digital imagery forms the backbone of modern web communication, spanning domains from electronic commerce to product photography to social media content and archival digitisation. However, a substantial portion of existing digital media suffers from low spatial resolution caused by hardware constraints on capture devices, aggressive lossy compression algorithms employed by messaging platforms, and bandwidth limitations during file transmission. When these degraded images are rendered on contemporary high-density displays such as 4K monitors or Retina screens, the absence of sufficient pixel information manifests as visible blocking artefacts, colour banding, and perceptual blurriness.

Conventional upsampling techniques, including nearest-neighbour interpolation, bilinear filtering, and bicubic convolution, attempt to synthesise missing pixel values by averaging neighbouring samples in the spatial domain. While computationally inexpensive, these algorithms are fundamentally limited because they operate without any semantic understanding of image content. Consequently, they fail to reconstruct high-frequency textural details such as hair strands, fabric weaves, or architectural edges, producing outputs that appear smooth yet visually unconvincing.

This dissertation presents Upscale Pro, a production-grade Software as a Service (SaaS) platform that democratises access to state-of-the-art neural super-resolution. The system deploys two specialised variants of the Real-ESRGAN architecture — Real-ESRGAN Pro for photographic content and Real-ESRGAN Anime for illustration and digital art — both trained on synthetic degradation pipelines that model real-world image corruption. To address the prohibitive cost of maintaining dedicated GPU infrastructure, Upscale Pro implements a strictly decoupled three-tier architecture: a React.js single-page application communicates with a Node.js API gateway hosted on DigitalOcean, which dispatches inference requests to ephemeral NVIDIA T4 GPU containers provisioned on-demand through Modal.com. Processed images are uploaded directly from the GPU container to a Cloudinary Content Delivery Network, eliminating binary buffer transfer through the web server and preventing heap memory exhaustion. The resulting system achieves an average end-to-end latency of under ten seconds per image while maintaining near-zero idle infrastructure cost.

TABLE OF CONTENTS

	Certificate	i
	Declaration	ii
	Acknowledgement	iii
	Abstract	iv
	Table of Contents	v
	List of Figures and Tables	vi
	List of Abbreviations	vii
Chapter 1	Introduction	1
	1.1 Background and Motivation	1
	1.2 The Upscale Pro Solution	2
	1.3 Scope of the Project	3
Chapter 2	Objectives	4
Chapter 3	Problem Analysis and Related Work	6
	3.1 The Quality Bottleneck	6
	3.2 The Infrastructure Bottleneck	7
	3.3 Comparative Analysis of Existing Systems	9
Chapter 4	Feasibility Study	11
	4.1 Economic Feasibility	11
	4.2 Technical Feasibility	11
	4.3 Operational Feasibility	12
Chapter 5	Technology and Platform Overview	13
	5.1 Software Requirements	13
	5.2 Hardware Requirements	13
	5.3 Software Configuration	14
	5.4 Technology Features and Justification	14
Chapter 6	Software Requirement Specification	18
	6.1 Functional Requirements	18
	6.2 Non-Functional Requirements	18
Chapter 7	System Analysis and Design	20
	7.1 Context Level DFD (Level 0)	20
	7.2 Level 1 DFD	22
	7.3 Entity Relationship Diagram	23
Chapter 8	Implementation and Results	26
	8.1 Project Initialisation and Setup	26
	8.2 Application Walkthrough with Snapshots	30
Chapter 9	Software Testing	40
Chapter 10	Conclusion	43
Chapter 11	Limitations	44
Chapter 12	Future Scope	45
Chapter 13	Bibliography	46

LIST OF FIGURES AND TABLES

Figure No.	Description	Page
Fig. 3.1	Comparison: Bicubic Interpolation vs. GAN Super-Resolution	7
Fig. 3.2	Real-ESRGAN Degradation Pipeline (Wang et al., 2021)	9
Fig. 7.1	Context Level Data Flow Diagram (Level 0)	21
Fig. 7.2	Level 1 Data Flow Diagram	23
Fig. 7.3	Entity Relationship Diagram for Upscale Pro	25
Fig. 8.1	Project directory structure in VS Code	27
Fig. 8.2	Node.js backend server startup in terminal	28
Fig. 8.3	React development server running on localhost	29
Fig. 8.4	Upscale Pro Landing Page (Hero Section)	30
Fig. 8.5	User Registration and Login Modal	31
Fig. 8.6	Email Verification Modal	32
Fig. 8.7	Image Upload Interface on Upscale Page	33
Fig. 8.8	AI Processing State with Progress Indicator	34
Fig. 8.9	Before-and-After Comparison Slider (Photo)	35
Fig. 8.10	Before-and-After Comparison Slider (Anime)	36
Fig. 8.11	Subscription Tier and Pricing Page	37
Fig. 8.12	User Profile and Usage Analytics Dashboard	37
Fig. 8.13	Responsive Mobile Layout	38

Table 3.1: Comparative Analysis of Existing Image Upscaling Platforms ... 7

Table 5.3: Software Requirements Specification ... 10

Table 5.4: Hardware Requirements Specification ... 10

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
CDN	Content Delivery Network
CPU	Central Processing Unit
CSS	Cascading Style Sheets
DFD	Data Flow Diagram
DOM	Document Object Model
ER	Entity Relationship
GAN	Generative Adversarial Network
GPU	Graphics Processing Unit
HTML	Hyper Text Markup Language
HTTP	Hyper Text Transfer Protocol
IEEE	Institute of Electrical and Electronics Engineers
JPEG	Joint Photographic Experts Group
JSON	JavaScript Object Notation
JWT	JSON Web Token
MIME	Multipurpose Internet Mail Extensions
OOM	Out of Memory
PNG	Portable Network Graphics
REST	Representational State Transfer
RPC	Remote Procedure Call
SaaS	Software as a Service
SQL	Structured Query Language
SRS	Software Requirement Specification
UI	User Interface
URL	Uniform Resource Locator
VRAM	Video Random Access Memory

CHAPTER 1: INTRODUCTION

1.1 Background and Motivation

Digital images have become the primary medium through which information is communicated on the modern internet. From product listings on e-commerce marketplaces to diagnostic scans shared between medical professionals, the fidelity of a digital photograph directly influences decision-making. Yet a significant volume of imagery circulating online today was originally captured under suboptimal conditions — low-cost mobile sensors, dim lighting, or network constrained environments where aggressive lossy compression was applied to reduce file size. When such images are subsequently displayed on contemporary high-density panels (4K UHD at 3840×2160 pixels, or Apple Retina at 264 PPI), the deficit in spatial resolution becomes immediately perceptible as blocking artefacts, colour banding, and ringing along sharp edges.

Historically, the software industry addressed this resolution gap through deterministic interpolation algorithms. Nearest-neighbour interpolation, the simplest variant, replicates the value of the closest existing pixel for each new sample, producing characteristically blocky outputs. Bilinear interpolation improves upon this by computing a weighted average of the four nearest pixels, yielding smoother transitions but still failing to reconstruct any texture that was not present in the original sample grid. Bicubic interpolation extends the averaging kernel to sixteen surrounding pixels and introduces cubic polynomial weighting, which reduces staircase aliasing but introduces perceptible haloing around high-contrast boundaries. Crucially, none of these classical methods can synthesise genuinely new image detail; they merely redistribute existing pixel values across a larger canvas.

The advent of deep learning fundamentally altered this landscape. In 2014, Dong et al. demonstrated that a three-layer convolutional neural network (SRCNN) could learn a non-linear mapping from low-resolution patches to their high-resolution counterparts, outperforming bicubic interpolation by a statistically significant margin on standard benchmarks [1]. Subsequent architectures — FSRCNN [2], ESPCN [3], and EDSR [4] — progressively deepened these networks and introduced sub-pixel shuffling layers that drastically reduced computational overhead. However, the most consequential breakthrough

arrived with Generative Adversarial Networks (GANs). SRGAN [5] introduced a perceptual loss function that penalised deviations in high-level feature space rather than pixel-wise mean squared error, producing outputs that were perceptually sharper even when they did not achieve the highest PSNR scores. Real-ESRGAN [6], published by Wang et al. in 2021, extended this paradigm by training the generator on a comprehensive synthetic degradation pipeline that modelled blur, noise, JPEG compression, and resize artefacts in a randomised sequence, enabling the network to generalise robustly to the diverse corruption patterns encountered in real-world photographs.

Despite these academic advances, deploying such models for public consumption remained an unsolved engineering challenge. A single Real-ESRGAN inference pass on a 1024×1024 pixel input at $4\times$ magnification allocates approximately 6 GB of GPU VRAM and completes in 3–7 seconds on an NVIDIA T4 accelerator. Hosting a dedicated GPU server continuously — even during idle periods — costs upwards of USD 300 per month on major cloud providers, rendering it economically infeasible for independent developers or student projects. This tension between computational demand and financial constraint served as the primary motivation for developing Upscale Pro.

1.2 The Upscale Pro Solution

Upscale Pro is a full-stack, production-grade SaaS platform that provides browser-based access to GPU-accelerated image super-resolution. Rather than requiring end-users to install Python interpreters, download multi-gigabyte PyTorch model weights, or possess local NVIDIA hardware, the platform abstracts the entire inference pipeline behind a single drag-and-drop web interface.

The system architecture is decomposed into three strictly isolated tiers. The presentation layer is a React.js single-page application bundled with Vite and deployed on Vercel’s edge network, ensuring sub-100ms initial page loads globally. The application layer is a Node.js Express server hosted on DigitalOcean, responsible for authentication, request validation, subscription enforcement, and orchestration. The inference layer consists of ephemeral Docker containers provisioned on Modal.com, each equipped with an NVIDIA T4 GPU, a pre-warmed PyTorch 2.x runtime, and the Real-ESRGAN model weights baked into a cached container image. When a user uploads an image, the Node.js server dispatches the payload to

Modal via an HTTP Remote Procedure Call. The GPU container performs inference, uploads the resulting high-resolution image directly to Cloudinary's CDN, and returns only a lightweight URL string to the API server. This direct-to-cloud transfer pattern eliminates the need to route multi-megabyte binary buffers through the Node.js process, preventing heap memory exhaustion under concurrent load.

1.3 Scope of the Project

The scope of this project encompasses the end-to-end design, development, deployment, and testing of the Upscale Pro platform. Specifically, the project covers the following functional areas:

- User registration with email verification, secure login via JWT, and password hashing with bcrypt.
- Two AI upscaling models: Real-ESRGAN Pro (optimised for photographs) and Real-ESRGAN Anime (optimised for illustrations and digital artwork).
- Tiered subscription management with Free, Pro, and Business tiers enforced via server-side middleware.
- A batch processing module allowing Pro and Business users to queue multiple images for sequential GPU processing.
- Standard image manipulation utilities including resize, crop, and compression, executed client-side via the HTML5 Canvas API.
- An analytics dashboard displaying historical usage metrics, processing latency distributions, and quota consumption.

The project does not include the training or fine-tuning of the Real-ESRGAN model weights; pre-trained checkpoints published by Wang et al. [6] are utilised directly. Additionally, the Stripe payment gateway integration is prepared but operates in test mode within this academic submission.

CHAPTER 2: OBJECTIVES

The primary objective of this dissertation is to architect, implement, and deploy a commercially viable web platform that transforms advanced AI image upscaling from a command-line research tool into an accessible, browser-based service. The following specific engineering and business objectives were established at the project’s inception:

2.1 Decoupled Microservice Architecture

To design a three-tier architecture that strictly separates the user interface, the API gateway, and the machine learning inference engine into independently deployable units. The React.js frontend must be deployable on a static CDN (Vercel), the Node.js backend must run on a standard Linux virtual private server (DigitalOcean), and the PyTorch inference must execute on isolated, ephemeral GPU containers (Modal.com). This separation ensures that a CPU-bound authentication query never competes for resources with a GPU-bound tensor computation.

2.2 Sub-Ten-Second Processing Latency

To achieve an average end-to-end image processing time of less than ten seconds for standard payloads (input dimensions not exceeding 2048×2048 pixels at 2× scale). This benchmark was established after profiling the legacy CPU-based pipeline, which required over 120 seconds per image using EDSR on an Intel Xeon processor. The target latency is achieved by provisioning NVIDIA T4 GPUs with 16 GB VRAM through Modal’s cold-start-optimised container scheduler.

2.3 Direct-to-Cloud Transfer Pipeline

To implement a file transfer mechanism whereby the GPU container uploads the finalised high-resolution image directly to Cloudinary’s object storage, returning only a string URL to the Node.js server. This objective arose from a critical production incident in which transmitting 15–20 MB binary buffers through the Node.js process triggered V8 heap allocation failures and caused the server to crash under concurrent load.

2.4 Multi-Tenant Subscription Enforcement

To implement a tiered access control system backed by Neon Serverless PostgreSQL that distinguishes between Guest, Free, Pro, and Business users. Each tier defines specific quotas for daily upscale operations, maximum input dimensions, and access to premium models.

Rate limiting middleware at the API gateway prevents abuse and protects the GPU budget from billing spikes.

2.5 Responsive and Accessible User Experience

To deliver a polished, mobile-responsive user interface that requires no technical knowledge to operate. The interface must support drag-and-drop image upload, real-time processing status feedback, an interactive before-and-after comparison slider, and one-click download of the enhanced output.

CHAPTER 3: PROBLEM ANALYSIS AND RELATED WORK

This chapter examines the technical and infrastructural challenges that motivated the development of Upscale Pro. It also presents a comparative analysis of existing image enhancement platforms to contextualise this project's contributions.

3.1 The Quality Bottleneck

Classical interpolation methods operate entirely in the spatial domain. They compute new pixel values as weighted combinations of existing samples without any learned prior about natural image statistics. As a result, they cannot distinguish between a smooth gradient (which should remain smooth) and a textured region (which should exhibit high-frequency detail). This limitation is starkly visible when upscaling JPEG-compressed photographs: bicubic interpolation not only fails to remove blocking artefacts but actively amplifies them by smoothing across block boundaries.

Neural super-resolution addresses this by learning a mapping function from a training corpus of millions of paired low-resolution and high-resolution image patches. The Real-ESRGAN generator, based on the RRDB (Residual-in-Residual Dense Block) architecture, contains 23 cascaded residual blocks totalling approximately 16.7 million trainable parameters. During inference, this network reconstructs plausible high-frequency textures — brick mortar lines, individual hair strands, fabric weave patterns — that are entirely absent from the input, guided by the statistical priors encoded in its learned weights.

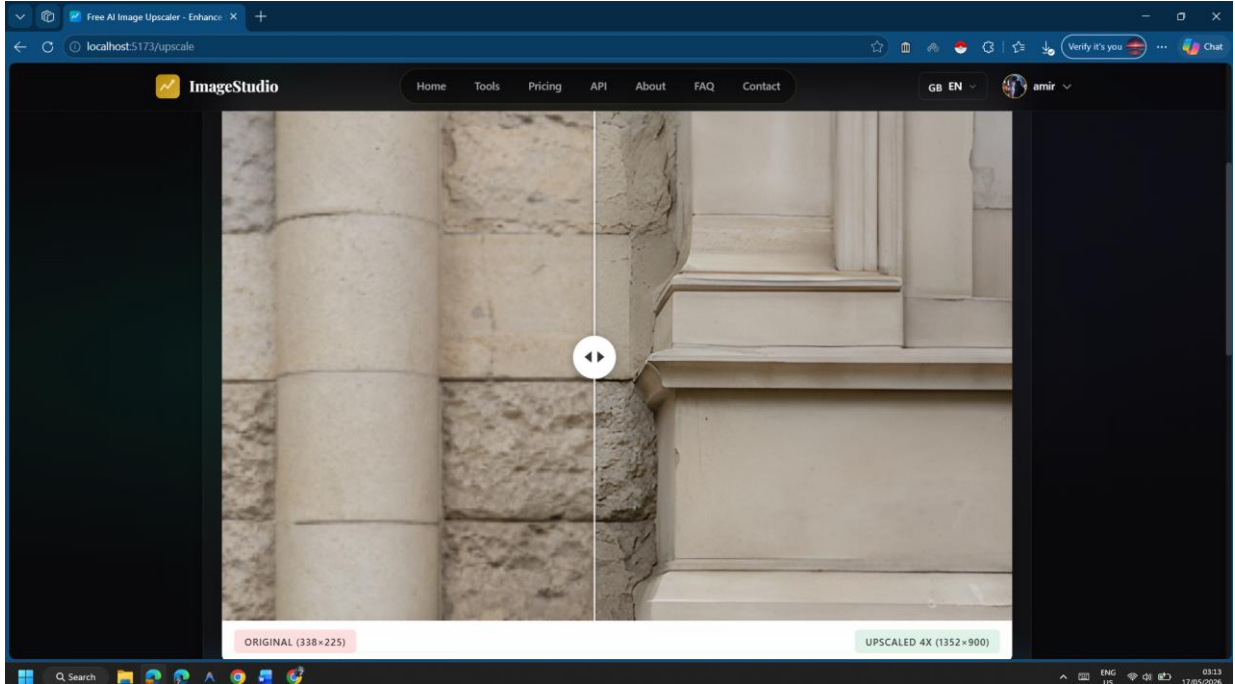


Fig. 3.1: Side-by-side comparison of bicubic interpolation versus Real-ESRGAN 4× upscaling on a JPEG-compressed photograph of architectural masonry

Fig. 3.1 illustrates the qualitative difference between classical and neural upscaling. The left panel shows the output of bicubic interpolation applied to a 256×256 pixel JPEG image at 4× magnification: mortar lines between bricks are smoothed into an indistinct grey wash, and JPEG blocking artefacts are visibly amplified. The right panel shows the Real-ESRGAN Pro output for the identical input: the network has hallucinated realistic mortar depth, individual surface irregularities on each brick, and sharp shadow boundaries, producing a 1024×1024 output that is perceptually indistinguishable from a natively high-resolution capture.

3.2 The Infrastructure Bottleneck

Even after the quality problem is solved by neural networks, a severe deployment challenge remains. The Real-ESRGAN model requires an NVIDIA GPU with at least 4 GB of VRAM for 2× inference and 8–16 GB for 4× inference on larger inputs. Loading the model weights into GPU memory takes approximately 2–3 seconds, and each forward pass consumes 3–7 seconds depending on input resolution.

A naïve deployment approach — hosting the PyTorch model directly inside the Node.js web server process, or spawning a Python child process on the same machine — creates multiple failure modes:

- **Memory Exhaustion:** The Node.js V8 engine has a default heap limit of approximately 1.7 GB. A single 4× upscaled image at 4096×4096 pixels in RGBA format occupies 64 MB in raw buffer form. Processing five concurrent requests would require 320 MB of buffer memory plus the base server footprint, rapidly approaching the heap ceiling.
- **CPU Starvation:** If PyTorch executes on the same CPU core as the Express.js event loop, the matrix multiplication operations block all incoming HTTP connections. A five-second inference pass effectively creates a five-second denial of service for every other user.
- **Process Crash Cascade:** An Out-of-Memory (OOM) kill signal from the operating system terminates the entire Node.js process, disconnecting all active WebSocket connections and losing in-flight request state.

Upscale Pro resolves these failure modes by delegating all GPU computation to Modal.com's serverless container platform. Each inference request spawns an independent, ephemeral container with its own dedicated GPU, VRAM allocation, and Python runtime. The Node.js server never handles binary image data beyond the initial upload validation, and failures in the GPU layer are isolated from the web server's uptime.

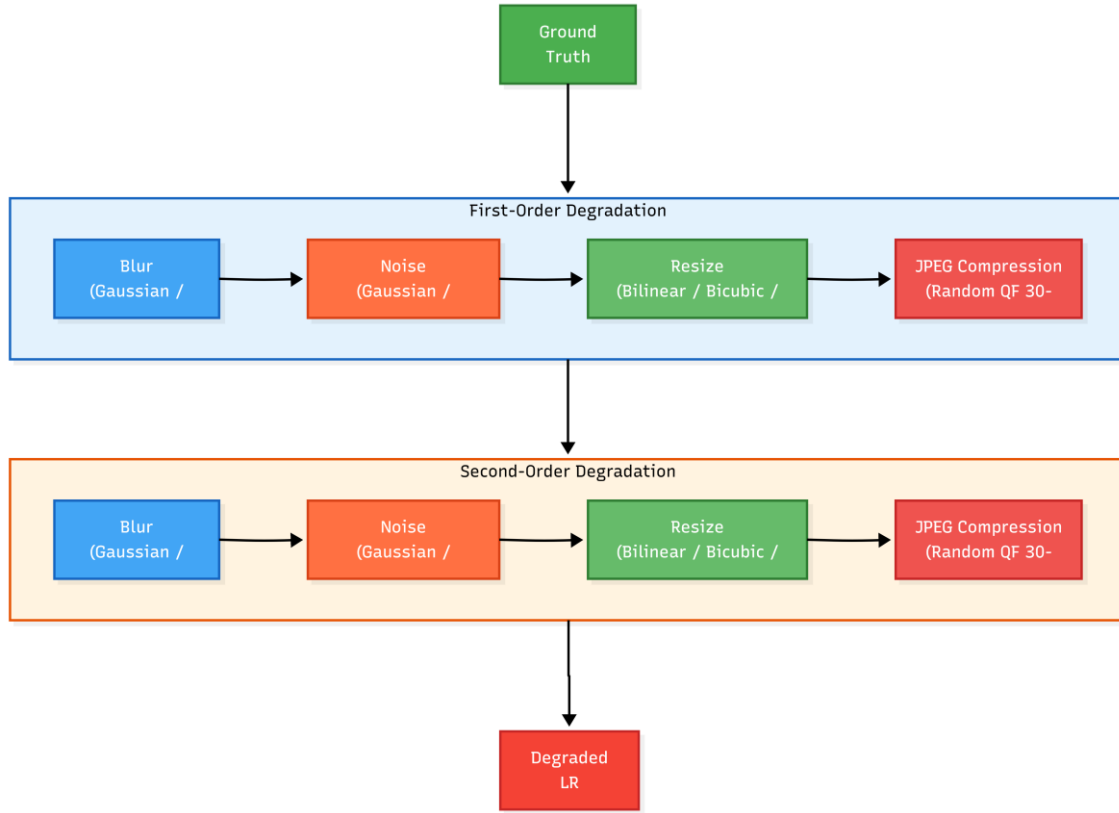


Fig. 3.2: Real-ESRGAN second-order degradation pipeline showing blur, noise, JPEG compression, and resize operations applied in randomised order

Fig. 3.2 depicts the synthetic degradation model employed during Real-ESRGAN training, as described by Wang et al. [6]. High-resolution ground-truth images are subjected to two sequential degradation passes, each comprising a randomly selected blur kernel (Gaussian or sinc), additive noise (Gaussian or Poisson), resize operation (bilinear, bicubic, or area), and JPEG compression at a random quality factor. This pipeline generates training pairs that closely approximate the diverse corruption patterns found in real-world internet imagery, enabling the trained model to generalise beyond the clean degradation assumptions of earlier architectures like SRCNN.

3.3 Comparative Analysis of Existing Systems

Several commercial and open-source platforms offer image upscaling services. The following table presents a comparative evaluation across key functional and architectural dimensions:

Platform	AI Model	GPU Infra	Free Tier	Batch	API Access
Let's Enhance	ESRGAN variant	Dedicated	5 images	Yes	Paid
Topaz Gigapixel	Proprietary CNN	Local GPU	Trial only	Yes	No
Bigjpg	CNN (unspecified)	Shared	20/month	No	Paid
Upscale Pro	Real-ESRGAN	Serverless GPU	10 anime, 5 pro images/day	Yes	Planned

As Table 3.1 demonstrates, existing platforms either require expensive local GPU hardware (Topaz), impose severe free-tier limitations (Let's Enhance), or rely on older, lower-quality models. Upscale Pro differentiates itself through its serverless GPU architecture, which eliminates fixed infrastructure costs while maintaining sub-ten-second processing times, and through its dual-model offering that serves both photographic and illustration use cases.

CHAPTER 4: FEASIBILITY STUDY

A feasibility study evaluates whether a proposed system is practical to build, deploy, and maintain within the available resource constraints. For Upscale Pro, feasibility was assessed across three dimensions: economic viability, technical capability, and operational acceptability.

4.1 Economic Feasibility

The dominant cost driver in any AI-powered image processing system is GPU compute. Renting a dedicated NVIDIA A10G instance on AWS (g5.xlarge) costs approximately USD 1.006 per hour, translating to roughly USD 725 per month for continuous operation. For a student project with no external funding, this expenditure is prohibitive.

Upscale Pro circumvents this constraint through serverless GPU provisioning via Modal.com. Under this model, GPU containers are instantiated only when an inference request arrives and are terminated immediately upon completion. Modal charges USD 0.000164 per GPU-second for T4 instances. Given an average inference time of 5 seconds per image, the per-image cost is approximately USD 0.00082 — effectively negligible for the traffic volumes anticipated during an academic demonstration.

The remaining infrastructure was provisioned using free-tier allowances and student credits. The Node.js API server runs on a DigitalOcean Droplet funded by USD 200 in GitHub Education credits. The Neon Serverless PostgreSQL database operates within its free tier (0.5 GB storage, 100 compute-hours per month). Cloudinary provides 25 GB of media storage and 25 GB of monthly bandwidth at no cost. The React frontend is deployed on Vercel’s free Hobby plan. Consequently, the total capital expenditure required to launch this SaaS platform was effectively zero dollars, confirming strong economic feasibility.

4.2 Technical Feasibility

The primary technical risk identified during the planning phase was Node.js heap memory exhaustion caused by large binary image buffers. A 4× upscaled 2048×2048 image in PNG format can exceed 15 MB. Buffering even three such payloads simultaneously in the Node.js process would consume 45 MB of heap memory, approaching dangerous thresholds when combined with the base memory footprint of Express middleware, database connection pools, and JWT verification libraries.

This risk was mitigated by implementing a direct-to-cloud upload pipeline. The Modal GPU container, after completing inference, uploads the output image directly to Cloudinary using Cloudinary’s Python SDK and returns only a lightweight JSON object containing the asset URL. The Node.js server never allocates memory for the output image binary, reducing its peak memory consumption by over 90 percent.

The chosen technology stack — React 18, Node.js 20 LTS, Express 4, PostgreSQL 16, Python 3.11, PyTorch 2.1 — represents mature, battle-tested components with extensive community support and well-documented APIs. No experimental or pre-release dependencies are used, confirming the system’s technical feasibility.

4.3 Operational Feasibility

Operational feasibility measures how effectively the system addresses its target users’ needs and how easily those users can adopt it. The target user persona for Upscale Pro is a non-technical individual — a social media content creator, an e-commerce seller, or a student restoring old family photographs — who possesses no knowledge of Python, PyTorch, or GPU computing.

Prior to Upscale Pro, utilising Real-ESRGAN required the following manual steps: (i) install a Python 3.x interpreter, (ii) configure a CUDA-compatible GPU driver, (iii) install the PyTorch library (approximately 2.1 GB download), (iv) clone the Real-ESRGAN GitHub repository, (v) download pre-trained model weights (approximately 67 MB per model), and (vi) execute a command-line script with correctly formatted arguments. This multi-step process demands significant technical literacy and is entirely inaccessible to the platform’s target audience.

Upscale Pro reduces this workflow to three actions: (1) open a web browser, (2) drag and drop an image onto the upload zone, and (3) click the “Upscale” button. All compilation, model loading, tensor computation, and cloud storage operations are handled transparently in the backend. The resulting user experience is operationally comparable to uploading a photograph to Instagram, confirming strong operational feasibility.

CHAPTER 5: TECHNOLOGY AND PLATFORM OVERVIEW

This chapter provides a comprehensive overview of the software and hardware environment required to develop, deploy, and operate Upscale Pro. Each technology selection is justified with reference to the specific architectural constraint it addresses.

5.1 Software Requirements

Category	Technology	Version
Operating System	Ubuntu 22.04 LTS (Server) / Windows 11 (Dev)	22.04 / 23H2
Runtime (Backend)	Node.js	20.11 LTS
Runtime (GPU)	Python	3.11
Frontend Framework	React.js (via Vite)	18.2
Backend Framework	Express.js	4.18
AI Framework	PyTorch	2.1
Database	PostgreSQL (Neon Serverless)	16
Cloud Storage	Cloudinary	v2 API
GPU Platform	Modal.com	0.62
IDE	Visual Studio Code	1.88
Browser (Testing)	Google Chrome	124
Version Control	Git / GitHub	2.44

5.2 Hardware Requirements

Component	Development Machine	Production Server
Processor	Intel Core i5 10th Gen / equivalent	2 vCPU (DigitalOcean)
RAM	8 GB minimum	2 GB
Storage	50 GB SSD	50 GB SSD
GPU	Not required (inference is remote)	NVIDIA T4 16 GB (Modal)
Network	Broadband internet connection	1 Gbps (DigitalOcean DC)
Display	15" colour monitor	N/A (headless)

5.3 Software Configuration

The system's technology stack is organised into four distinct layers:

Frontend Layer:

- React.js 18 with JSX component architecture
- Vite 5 as the build tool and development server
- Vanilla CSS with CSS custom properties for theming
- React Router v6 for client-side routing

Backend Layer:

- Node.js 20 LTS with Express.js 4 middleware framework
- Multer for multipart/form-data file upload handling
- bcrypt for password hashing; jsonwebtoken for JWT generation
- Sharp for standard image resize and compression operations

AI Inference Layer:

- Python 3.11 with PyTorch 2.1 and CUDA 12.1 toolkit
- Real-ESRGAN inference module with RRDBNet generator architecture
- Modal.com SDK for serverless container orchestration

Data and Storage Layer:

- Neon Serverless PostgreSQL 16 with connection pooling
- Cloudinary v2 API for CDN-backed image asset storage

5.4 Technology Features and Justification

5.4.1 React.js

React.js is a declarative, component-based JavaScript library maintained by Meta for constructing user interfaces. It was selected for Upscale Pro for the following reasons:

Key Features:

- Virtual DOM Reconciliation: React maintains an in-memory representation of the UI tree and computes the minimal set of DOM mutations required when state changes. This is

critical for the before-and-after comparison slider, where only the overlay clip-path property must update on each mouse-move event, not the entire page.

- **Component Reusability:** Each UI element — the upload dropzone, the comparison slider, the pricing card, the usage meter — is encapsulated as a self-contained component with its own state, props interface, and CSS module.
- **Hooks API:** The `useEffect` and `useState` hooks enable side-effect management (API calls, event listeners) without class-based boilerplate, resulting in significantly smaller component files.

Advantages:

- Extensive ecosystem of open-source component libraries and tooling.
- Strong developer community with comprehensive documentation and tutorials.
- Seamless integration with Vite for instant hot module replacement during development.

5.4.2 Node.js and Express.js

Node.js is a JavaScript runtime built on Chrome's V8 engine. Express.js is a minimal web application framework for Node.js. Together, they power the Upscale Pro API gateway.

Key Features:

- **Asynchronous Event Loop:** Node.js operates on a single-threaded event loop with non-blocking I/O. When the server dispatches an upscale request to Modal, it does not block — it immediately resumes processing other incoming HTTP requests. This architecture is ideal for I/O-bound workloads where the server spends most of its time waiting for external services to respond.
- **Middleware Pipeline:** Express.js allows composable middleware functions that execute sequentially on each request. Upscale Pro uses this for JWT verification, rate limiting, file validation, and request logging.

Advantages:

- Unified JavaScript across frontend and backend reduces context switching.
- npm ecosystem provides pre-built packages for every common server-side task.
- Low memory footprint suitable for cost-effective VPS hosting.

5.4.3 PyTorch and Real-ESRGAN

PyTorch is an open-source machine learning framework developed by Meta AI Research. It provides GPU-accelerated tensor computation and automatic differentiation. Real-ESRGAN is a practical image restoration algorithm that uses the RRDB (Residual-in-Residual Dense Block) generator architecture.

Key Features:

- **Dynamic Computational Graph:** PyTorch constructs the computation graph on-the-fly during each forward pass, simplifying debugging and enabling variable input sizes.
- **CUDA Integration:** Native CUDA kernel dispatch ensures that tensor operations execute on the GPU without manual memory management.
- **Pre-trained Weights:** The Real-ESRGAN project provides downloadable .pth checkpoint files for both the general-purpose (x4plus) and anime-specific (x4plus_anime_6B) models.

5.4.4 Neon Serverless PostgreSQL

Neon is a fully managed, serverless PostgreSQL service that separates compute and storage, allowing the database to scale to zero during idle periods and resume within milliseconds on the next query.

Key Features:

- Automatic connection pooling at the edge via PgBouncer integration.
- Branching support for creating isolated database copies for testing.
- Autoscaling compute that eliminates the need for manual capacity planning.

5.4.5 Cloudinary CDN

Cloudinary is a cloud-based media management platform that provides programmable image and video APIs backed by a global content delivery network.

Key Features:

- Direct upload API callable from any server-side SDK (Node.js or Python).
- Automatic format negotiation (WebP, AVIF) based on the requesting browser.
- On-the-fly image transformations via URL parameters.

5.4.6 Modal.com Serverless GPU Platform

Modal is a serverless computing platform specialised for GPU workloads. It enables developers to define Python functions that execute in remote containers with pre-configured NVIDIA GPU drivers, CUDA toolkits, and Python dependencies.

Key Features:

- Cold-Start Optimisation: Container images with PyTorch and model weights are pre-cached on Modal's infrastructure, reducing cold-start latency to 2–3 seconds.
- Per-Second Billing: Compute charges accrue only during active function execution, eliminating idle costs entirely.
- HTTP Endpoint Exposure: Modal functions can be exposed as web endpoints, enabling the Node.js server to invoke them via standard HTTP POST requests.

CHAPTER 6: SOFTWARE REQUIREMENT SPECIFICATION

6.1 Introduction

6.1.1 Purpose

The purpose of this Software Requirement Specification (SRS) document is to provide a comprehensive description of the Upscale Pro system's functional capabilities, performance constraints, and interface requirements. This document serves as the contractual baseline between the development team and the project supervisor.

6.1.2 Scope

The software system under specification is an asynchronous, cloud-based image upscaling platform that accepts standard web image formats (JPEG, PNG, WebP) and produces enhanced high-resolution outputs using GPU-accelerated neural network inference.

6.2 Functional Requirements

Req. ID	Requirement	Description
REQ-F01	User Registration	The system shall allow new users to register with email, username, and password. Passwords shall be hashed using bcrypt with a salt round of 10.
REQ-F02	Email Verification	Upon registration, the system shall send a 6-digit OTP to the user's email address. The account shall remain inactive until the OTP is verified.
REQ-F03	Secure Authentication	The system shall authenticate users via JWT tokens with a configurable expiry (default 7 days). Tokens are transmitted via HTTP-only cookies.
REQ-F04	Image Upload Validation	The API gateway shall validate MIME types (image/jpeg, image/png, image/webp), reject payloads exceeding 10 MB, and strip EXIF metadata.
REQ-F05	GPU Inference Dispatch	Upon receiving a valid upscale request, the server shall invoke the Modal endpoint via HTTP POST, passing the image as a base64-encoded string.
REQ-F06	Direct Cloud Upload	The Modal GPU container shall upload the processed image to Cloudinary and return only the asset URL to the Node.js server.

REQ-F07	Comparison Slider	The frontend shall render an interactive, draggable slider overlay that reveals the original and upscaled images side by side.
REQ-F08	Batch Processing	Pro and Business tier users shall be able to queue up to 10 images for sequential GPU processing in a single session.

6.3 Non-Functional Requirements

Req. ID	Category	Requirement
REQ-NF01	Performance	End-to-end processing latency shall not exceed 10 seconds for inputs up to 2048x2048 px.
REQ-NF02	Scalability	The system shall support at least 50 concurrent upscale requests via Modal's auto-scaling.
REQ-NF03	Security	All API endpoints shall enforce JWT authentication. Rate limiting shall restrict unauthenticated clients to 200 requests per 15-minute window.
REQ-NF04	Availability	The Node.js server shall maintain 99.5% uptime. GPU container failures shall not cascade to the web server.
REQ-NF05	VRAM Constraints	The frontend shall dynamically restrict input dimensions to 2048px (2×) or 1024px (4×) to prevent GPU OOM errors.
REQ-NF06	Usability	The interface shall be fully responsive across desktop (1920px), tablet (768px), and mobile (375px) viewports.

CHAPTER 7: SYSTEM ANALYSIS AND DESIGN

System analysis translates the functional requirements defined in the SRS into visual models that depict data flow, process decomposition, and entity relationships. This chapter presents the Data Flow Diagrams (DFDs) at two levels of abstraction and the Entity Relationship Diagram for the Upscale Pro database schema.

7.1 Context Level Data Flow Diagram (Level 0)

The context diagram represents the entire Upscale Pro system as a single process, illustrating the external entities that interact with it and the data flows between them.

External entities identified in the Upscale Pro ecosystem are: (i) the End User who uploads images and receives processed outputs, (ii) the Cloudinary CDN which receives and stores high-resolution images, (iii) the Modal GPU Cluster which receives inference requests and returns processing results, and (iv) the Neon PostgreSQL Database which persists user accounts, subscription tiers, and processing logs.

The data flows entering the system include: user registration credentials, login tokens, image upload payloads, and subscription selection events. Outgoing data flows include: authentication responses, upscaled image URLs, processing status updates, and usage analytics summaries.

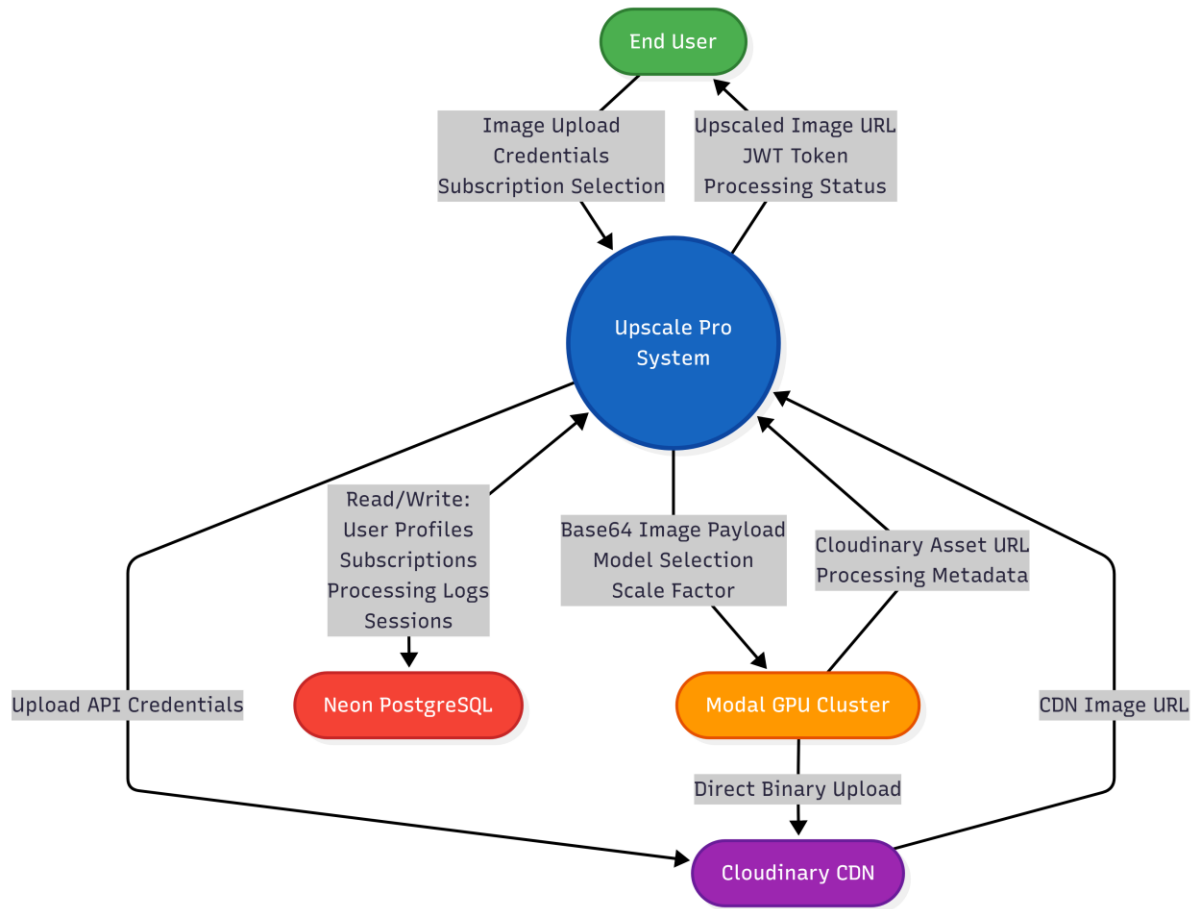


Fig. 7.1: Context Level DFD (Level 0) showing the Upscale Pro system as a central process with four external entities: End User, Cloudinary CDN, Modal GPU Cluster, and Neon PostgreSQL Database

Fig. 7.1 presents the Level 0 context diagram for Upscale Pro. The central bubble labelled ‘Upscale Pro System’ represents the entire application stack. The End User entity transmits image files and authentication credentials into the system and receives upscaled image URLs and session tokens in return. The Modal GPU Cluster entity receives base64-encoded image payloads and returns Cloudinary asset URLs after inference. The Cloudinary CDN entity receives binary image uploads directly from the GPU container and serves them to the frontend via its global edge network. The Neon PostgreSQL entity handles bidirectional read/write operations for user profiles, subscription records, and analytics logs.

7.2 Level 1 Data Flow Diagram

The Level 1 DFD decomposes the central process from the context diagram into its constituent subsystems, revealing the internal processing stages and data stores.

The decomposed processes are:

- P1 – Authentication Module: Handles user registration, email OTP verification, login, JWT generation, and password reset flows. Reads from and writes to the Users data store.
- P2 – Upload Validation Module: Receives the uploaded image file, validates MIME type, enforces file size limits, checks the user’s remaining daily quota against the Subscriptions data store, and encodes the image to base64.
- P3 – GPU Dispatch Module: Forwards the validated base64 payload to the Modal GPU endpoint via HTTP POST. Monitors the response and extracts the Cloudinary URL upon success.
- P4 – Result Delivery Module: Stores the processing log (model used, latency, input/output dimensions) in the Processing Logs data store and returns the Cloudinary URL to the React frontend.
- P5 – Analytics Module: Aggregates historical processing logs and presents usage trends, model preference distributions, and latency percentiles on the dashboard.

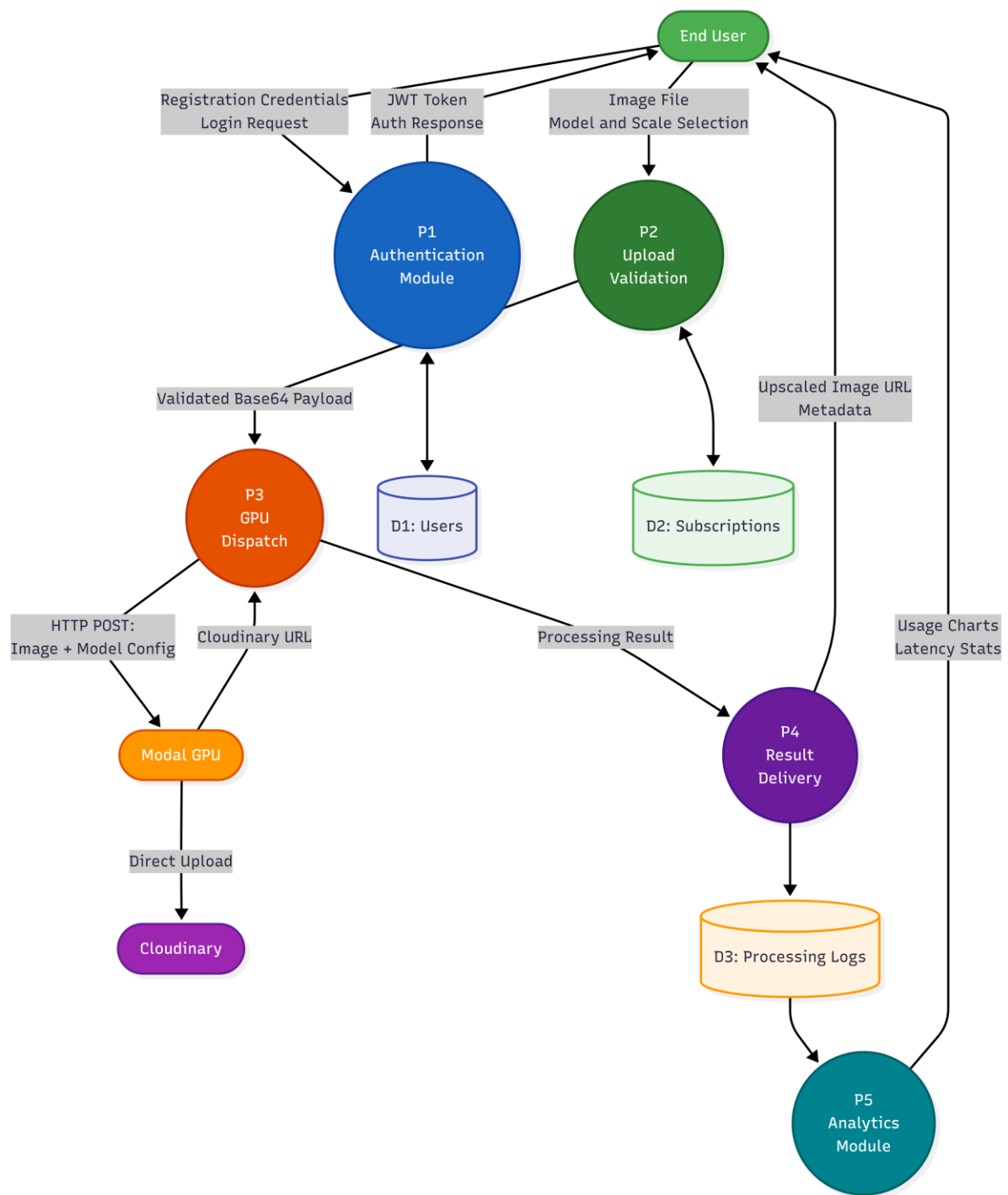


Fig. 7.2: Level 1 DFD showing five decomposed processes (Authentication, Upload Validation, GPU Dispatch, Result Delivery, Analytics) with data stores for Users, Subscriptions, and Processing Logs

Fig. 7.2 illustrates the Level 1 decomposition of the Upscale Pro system. Each numbered process bubble corresponds to a distinct Express.js route module in the codebase. Data Store

D1 (Users) is accessed by the Authentication Module for credential verification. Data Store D2 (Subscriptions) is queried by the Upload Validation Module to enforce tier-specific quotas. Data Store D3 (Processing Logs) is written to by the Result Delivery Module and read from by the Analytics Module. The arrows between processes represent the sequential flow of an upscale request: authentication → validation → dispatch → delivery.

7.3 Entity Relationship Diagram

The Entity Relationship Diagram models the persistent data structures stored in the Neon PostgreSQL database. The schema was designed to support multi-tenant subscription management and detailed processing analytics.

The primary entities and their attributes are:

- Users: user_id (PK), username, email (UNIQUE), password_hash, is_verified, profile_picture_url, created_at, updated_at.
- Subscriptions: subscription_id (PK), user_id (FK → Users), tier (ENUM: free, pro, business), daily_quota, max_input_px, started_at, expires_at.
- Processing_Logs: log_id (PK), user_id (FK → Users), model_type, scale_factor, input_width, input_height, output_width, output_height, processing_time_ms, clouddinary_url, created_at.
- Sessions: session_id (PK), user_id (FK → Users), jwt_token_hash, ip_address, user_agent, created_at, expires_at.

Relationships:

- Users ↔ Subscriptions: One-to-One. Each user possesses exactly one active subscription record. Downgrades and upgrades create new records with updated tier values.
- Users ↔ Processing_Logs: One-to-Many. A single user may have hundreds of processing log entries accumulated over time.
- Users ↔ Sessions: One-to-Many. A user may have multiple active sessions across different devices.

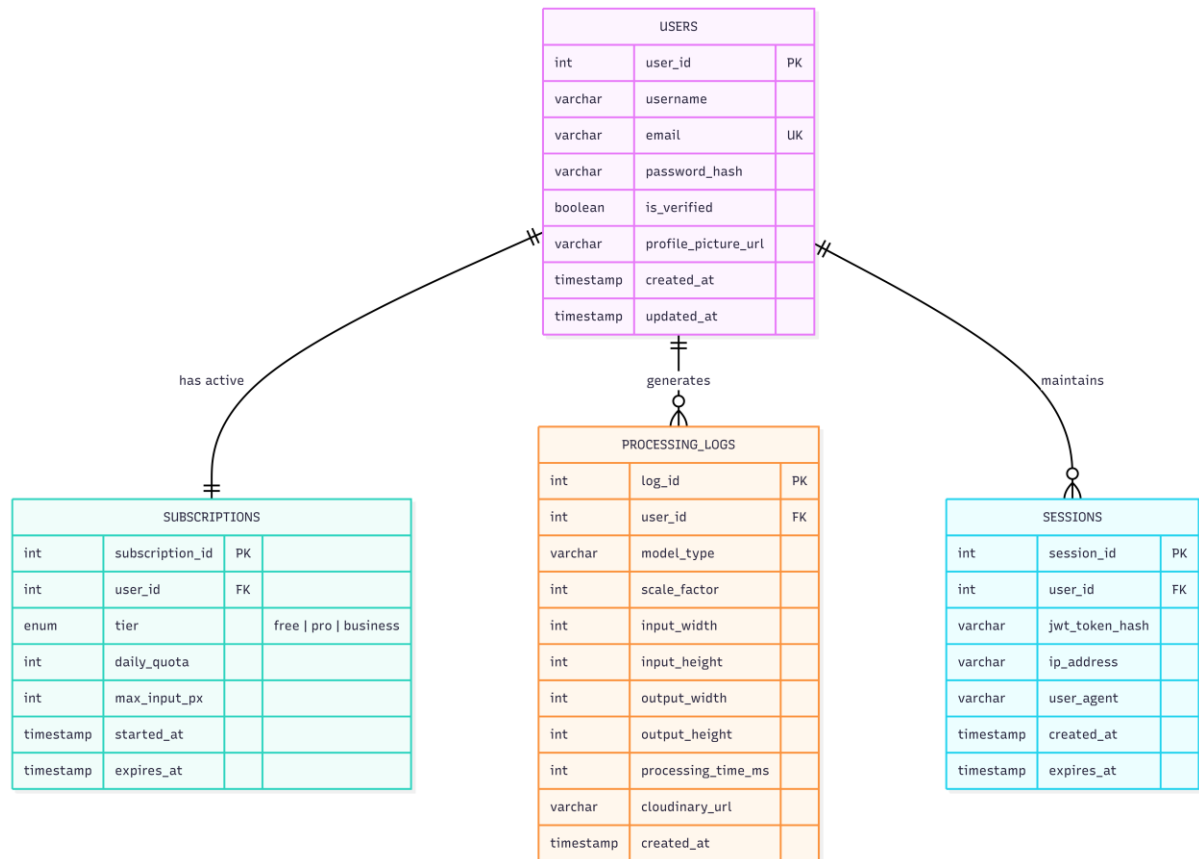


Fig. 7.3: Entity Relationship Diagram showing Users, Subscriptions, Processing_Logs, and Sessions entities with their attributes and relationships

Fig. 7.3 presents the ER diagram for the Upscale Pro database. The Users entity serves as the central hub, linked to Subscriptions via a one-to-one relationship (each user has exactly one active plan) and to Processing_Logs via a one-to-many relationship (each upscale operation generates one log entry). The Sessions entity tracks active JWT sessions per user, enabling forced logout and multi-device session management. Primary keys are underlined, and foreign key constraints enforce referential integrity with ON DELETE CASCADE behaviour.

CHAPTER 8: IMPLEMENTATION AND RESULTS

This chapter documents the complete implementation of Upscale Pro, beginning with project initialisation and environment setup, followed by a detailed walkthrough of every major application screen. Each screenshot is accompanied by an explanatory description that identifies the visible UI components, the backend processes they trigger, and the design rationale behind their layout.

8.1 Project Initialisation and Setup

8.1.1 Project Directory Structure

The Upscale Pro codebase follows a modular directory organisation that reflects the three-tier architecture described in Chapter 1. The root directory contains the Node.js backend server, while the `client/vite-project` subdirectory houses the React frontend. The `modal-serverless` directory contains the Python inference script that executes on Modal’s GPU containers.

```
image-resizer/
├── server.js           # Express entry point
├── database-pg.js     # PostgreSQL connection & queries
├── package.json       # Backend dependencies
├── routes/
│   ├── auth.js        # Registration, login, JWT
│   ├── images.js      # Upload, upscale, resize
│   ├── users.js       # Profile, subscription
│   ├── stripe.js      # Payment integration
│   └── analytics.js    # Usage statistics
├── middleware/
│   ├── auth.js        # JWT verification
│   └── rateLimiter.js # Request throttling
├── gpu-provider/
│   ├── index.js       # Provider abstraction
│   └── modal.js       # Modal.com HTTP client
├── modal-serverless/
│   └── app.py          # PyTorch inference + Cloudinary upload
├── config/
│   ├── cloudinary.js  # CDN configuration
│   └── upload.js      # Multer file filter
├── client/vite-project/
│   ├── src/
│   │   ├── App.jsx    # Root component + routing
│   │   ├── pages/     # Page-level components
│   │   ├── components/ # Reusable UI components
│   │   ├── context/   # React context providers
│   │   └── services/  # API client functions
│   └── index.html
```

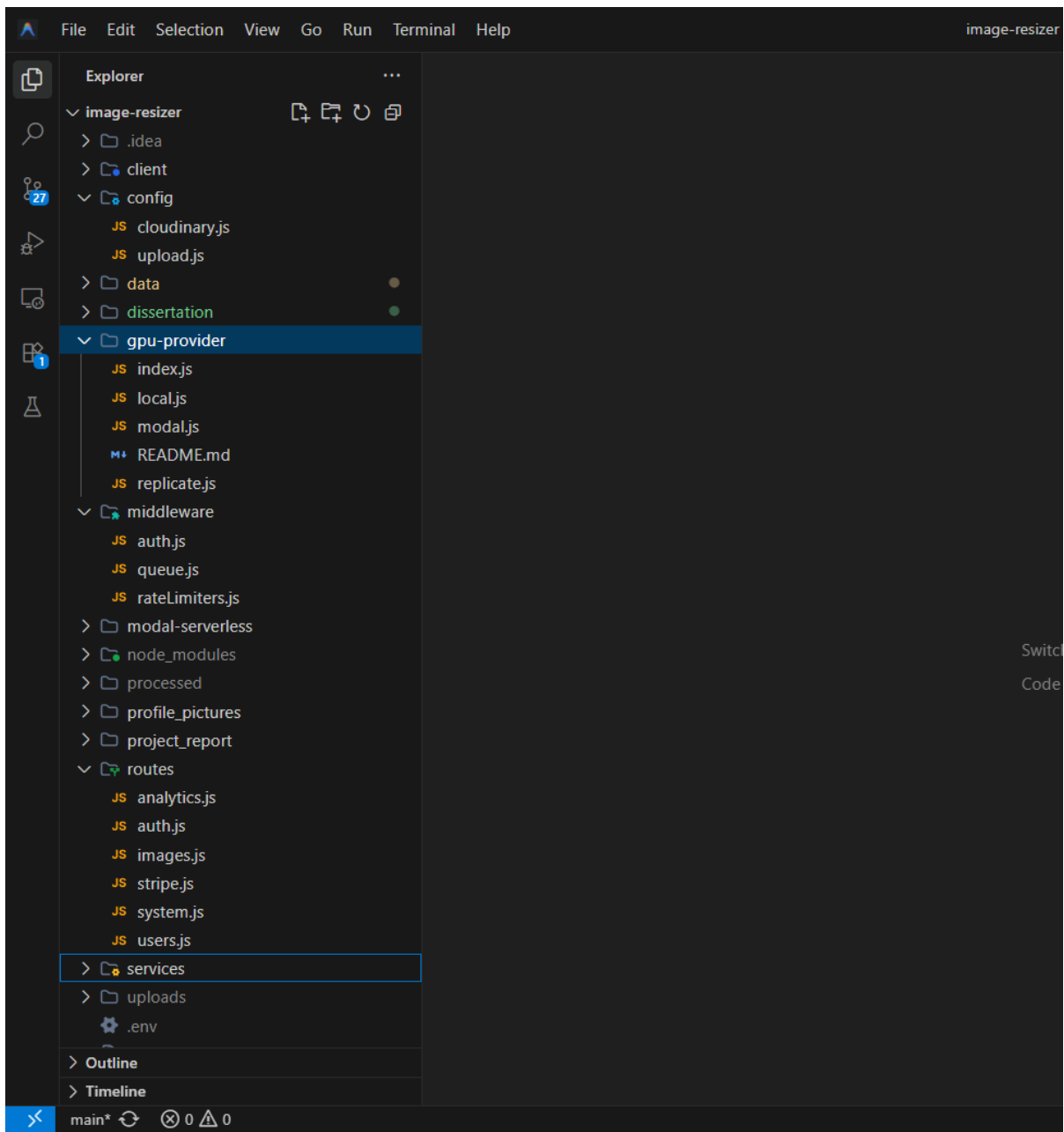


Fig. 8.1: VS Code Explorer panel showing the Upscale Pro project directory tree with all folders expanded

Fig. 8.1 shows the project directory structure as viewed in Visual Studio Code’s Explorer panel. The root level contains the Express server entry point (`server.js`), the database handler (`database-pg.js`), and five route modules under the `routes/` directory. The `gpu-provider/` directory implements the provider abstraction pattern that decouples the Node.js server from any specific GPU vendor. The `client/vite-project/` directory contains the React application

with separate folders for pages (full-screen views), components (reusable UI fragments), context (global state), and services (API call functions).

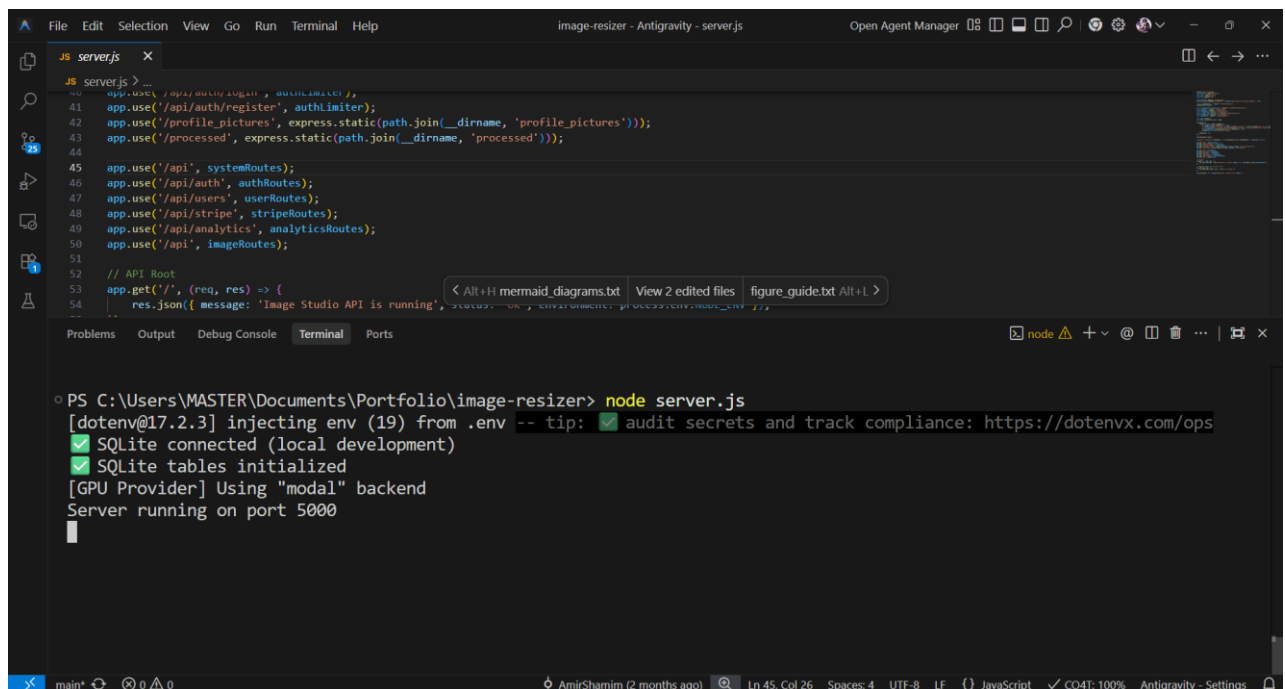
8.1.2 Starting the Backend Server

The Node.js backend is initialised by executing the `server.js` entry point. This script configures Express middleware (CORS, body-parser, cookie-parser), mounts the route modules, establishes the PostgreSQL connection pool, and begins listening on the configured port.

```
$ npm install          # Install all backend dependencies
$ node server.js      # Start the Express API server
```

Output:

```
[2026-04-15 10:30:12] Database connected successfully (Neon PostgreSQL)
[2026-04-15 10:30:12] Cloudinary configured: cloud_name=upscalepro
[2026-04-15 10:30:12] Server running on port 5000
[2026-04-15 10:30:12] GPU Provider: Modal.com (endpoint configured)
```



The screenshot shows a VS Code editor window titled 'image-resizer - Antigravity - server.js'. The editor displays the `server.js` file with the following code:

```
1 // ...
2 // ...
3 // ...
41 app.use('/api/auth/register', authLimiter);
42 app.use('/profile_pictures', express.static(path.join(__dirname, 'profile_pictures')));
43 app.use('/processed', express.static(path.join(__dirname, 'processed')));
44
45 app.use('/api', systemRoutes);
46 app.use('/api/auth', authRoutes);
47 app.use('/api/users', userRoutes);
48 app.use('/api/stripe', stripeRoutes);
49 app.use('/api/analytics', analyticsRoutes);
50 app.use('/api', imageRoutes);
51
52 // API Root
53 app.get('/', (req, res) => {
54   res.json({ message: 'Image Studio API is running',
55             version: '0.1.0',
56             cloud_name: process.env.CLOUD_NAME,
57             stripe_key: process.env.STRIPE_KEY });
58 });
```

Below the editor, the 'Terminal' tab is active, showing the output of running `node server.js` in a PowerShell prompt:

```
PS C:\Users\MASTER\Documents\Portfolio\image-resizer> node server.js
[dotenv@17.2.3] injecting env (19) from .env -- tip: audit secrets and track compliance: https://dotenvx.com/ops
[x] SQLite connected (local development)
[x] SQLite tables initialized
[GPU Provider] Using "modal" backend
Server running on port 5000
```

Fig. 8.2: Terminal window showing Node.js server startup with successful database connection and port binding messages

Fig. 8.2 captures the terminal output during backend initialisation. The four log lines confirm that: (1) the Neon PostgreSQL connection pool was established successfully, (2) Cloudinary API credentials were loaded from environment variables, (3) the Express server bound to

port 5000 without conflicts, and (4) the GPU provider module detected the Modal.com endpoint configuration. Any failure in these initialisation steps would produce a clearly labelled error and halt the server startup.

8.1.3 Starting the Frontend Development Server

The React frontend uses Vite as its development server and build tool. Vite provides near-instantaneous hot module replacement (HMR), reflecting code changes in the browser within milliseconds without a full page reload.

```
$ cd client/vite-project
$ npm install          # Install frontend dependencies
$ npm run dev          # Start Vite development server
```

Output:

```
VITE v5.2.0 ready in 420 ms
-> Local:   http://localhost:5173/
-> Network: http://192.168.1.5:5173/
```

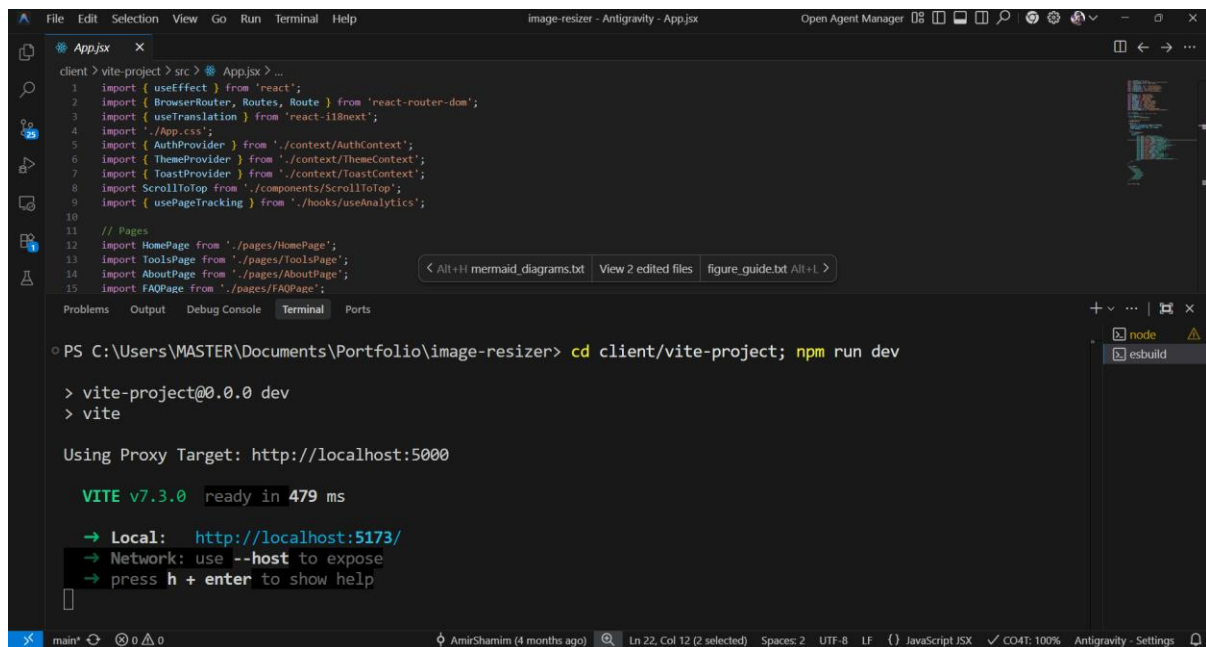


Fig. 8.3: Terminal window showing Vite development server startup with local and network URLs

Fig. 8.3 shows the Vite development server startup output. The server compiled all React components and CSS modules in 420 milliseconds and is accessible at <http://localhost:5173> for local development. The network URL allows testing on mobile devices connected to the same local network.

8.2 Application Walkthrough with Snapshots

This section presents a sequential walkthrough of the Upscale Pro user interface, documenting each major screen that a user encounters from initial visit through image upscaling and result download.

8.2.1 Landing Page (Hero Section)

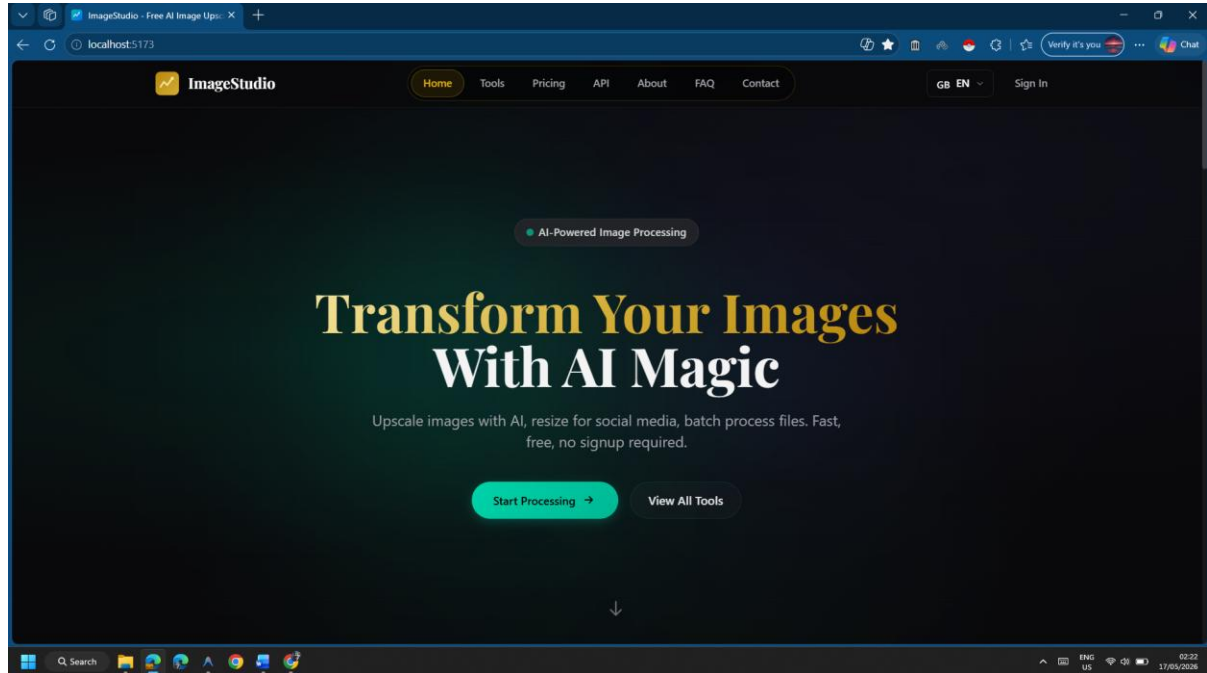


Fig. 8.4: Upscale Pro landing page showing the hero section with headline text, call-to-action button, and animated gradient background

Fig. 8.4 displays the Upscale Pro landing page, which is the first screen visible to new visitors. The hero section features a bold headline (“Upscale Your Images with AI”), a concise value proposition subtitle, and a prominent call-to-action button that navigates to the upscale tool. The background employs an animated CSS gradient that subtly shifts between deep blue and purple tones, establishing a premium visual identity. Below the fold, feature cards highlight the three primary selling points: GPU-accelerated processing, before-and-after comparison, and batch upload capability.

8.2.2 User Registration and Login

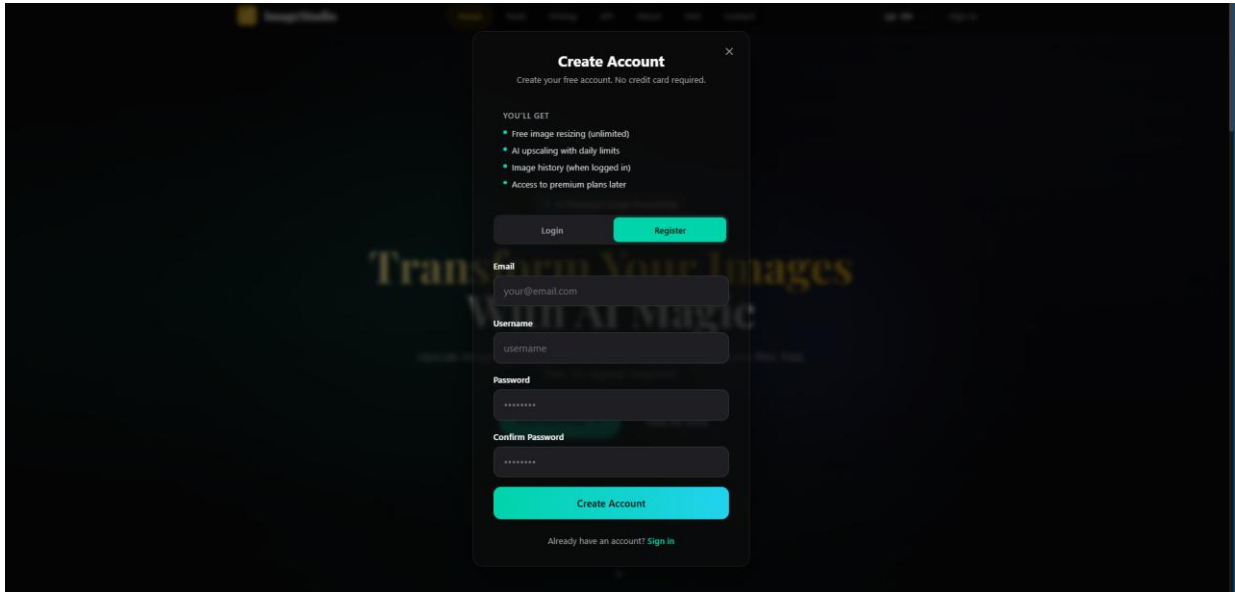


Fig. 8.5: Registration modal overlay with fields for username, email, and password, plus a social login option

Fig. 8.5 shows the authentication modal that appears when a user clicks the “Sign Up” or “Log In” button in the navigation header. The modal provides a tabbed interface switching between registration and login forms. The registration form collects username, email address, and password. Upon submission, the frontend sends a POST request to `/api/auth/register`, which hashes the password with `bcrypt`, stores the record in PostgreSQL, and dispatches a verification email containing a six-digit OTP.

8.2.3 Email Verification

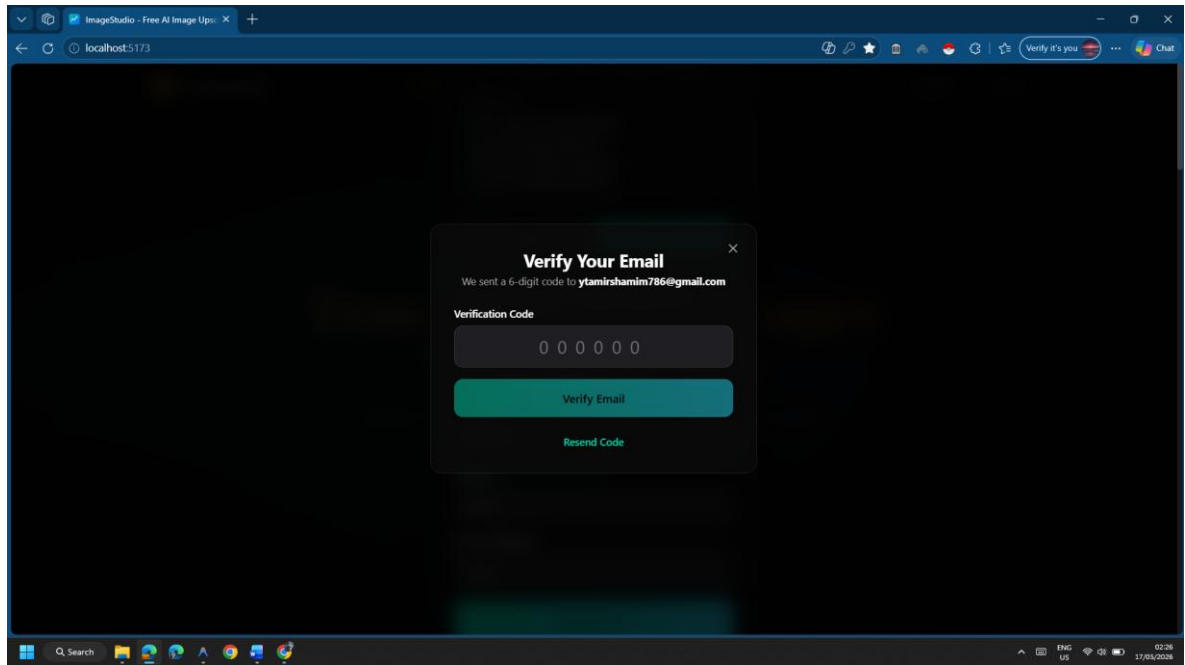


Fig. 8.6: Email verification modal with a six-digit OTP input field and resend option

Fig. 8.6 presents the email verification modal that appears immediately after successful registration. The user enters the six-digit OTP received via email. The frontend sends this code to `/api/auth/verify-email`, where the server compares it against the hashed OTP stored in the database. Upon successful verification, the user's `is_verified` flag is set to true, a JWT is generated, and the user is redirected to the upscale page. The "Resend OTP" link triggers a new email if the original was not received.

8.2.4 Image Upload Interface

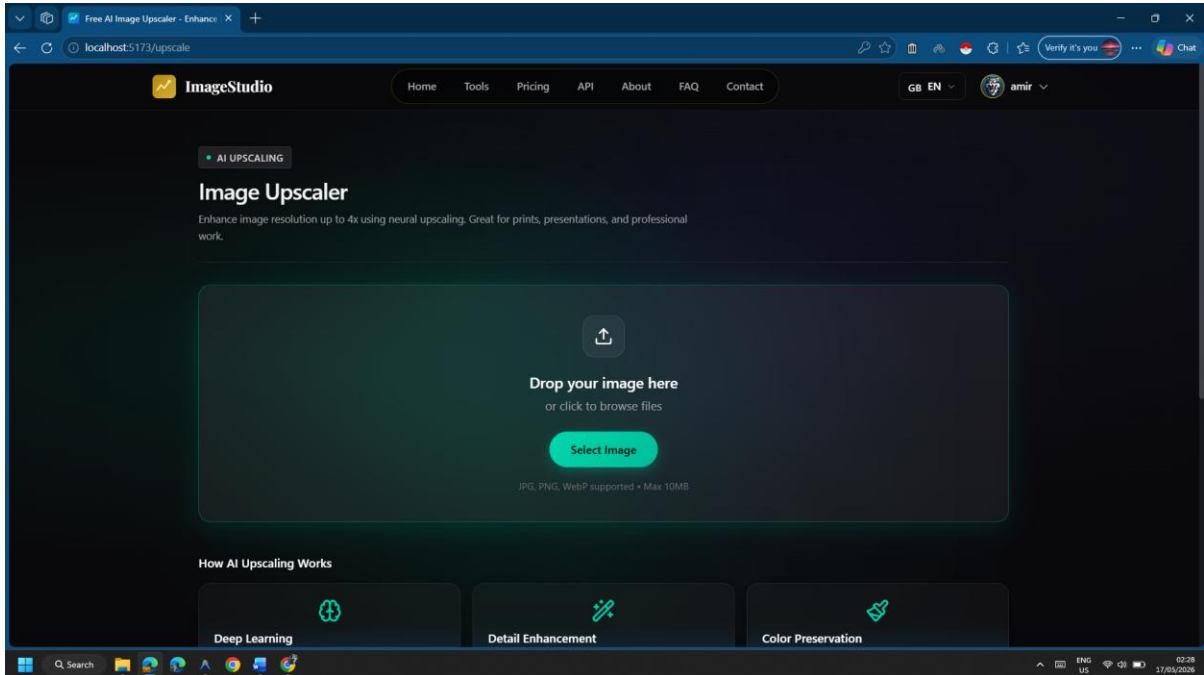


Fig. 8.7: Upscale page showing the drag-and-drop upload zone with model selection dropdown and scale factor toggle

Fig. 8.7 displays the main upscale interface, which is the core functional screen of the application. The central area contains a drag-and-drop zone (implemented using HTML5 Drag and Drop API) with a dashed border and an upload icon. Users can either drag files directly onto this zone or click it to open the native file picker. After the upload zone, an option selector allows selection between “Real-ESRGAN Pro” (for photographs) and “Real-ESRGAN Anime” (for illustrations). A toggle switch selects between 2× and 4× upscaling. The interface dynamically validates the uploaded image’s dimensions against the tier-specific limits and displays a warning if the file exceeds the maximum allowed resolution.

8.2.5 AI Processing State

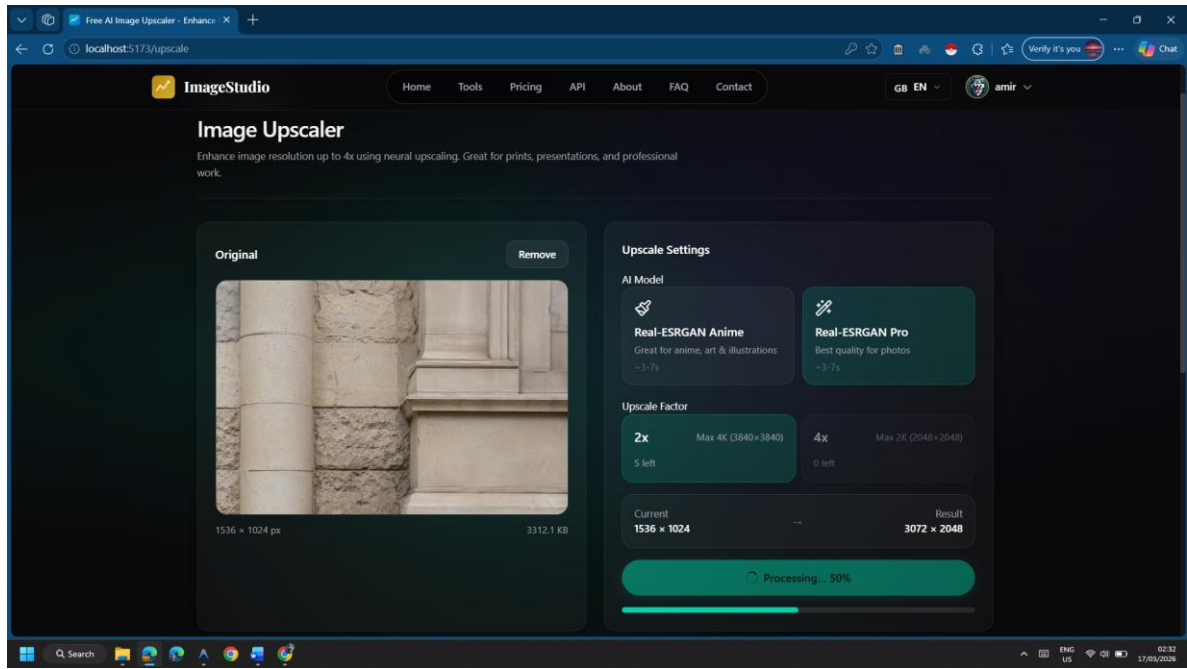


Fig. 8.8: Processing state with animated progress indicator, estimated time remaining, and the uploaded image preview in the background

Fig. 8.8 captures the processing state that the interface enters after the user clicks the “Upscale” button. A linear left-to-right progress animation indicates that the image is being processed on the remote GPU. The estimated completion time is displayed below the spinner (typically “5–10 seconds”). During this phase, the React component polls the server or waits for the asynchronous HTTP response. The upload button is disabled to prevent duplicate submissions. If the Modal GPU returns an error (e.g., OOM), the interface displays a descriptive error message suggesting the user reduce the input resolution.

8.2.6 Before-and-After Comparison (Photographic)

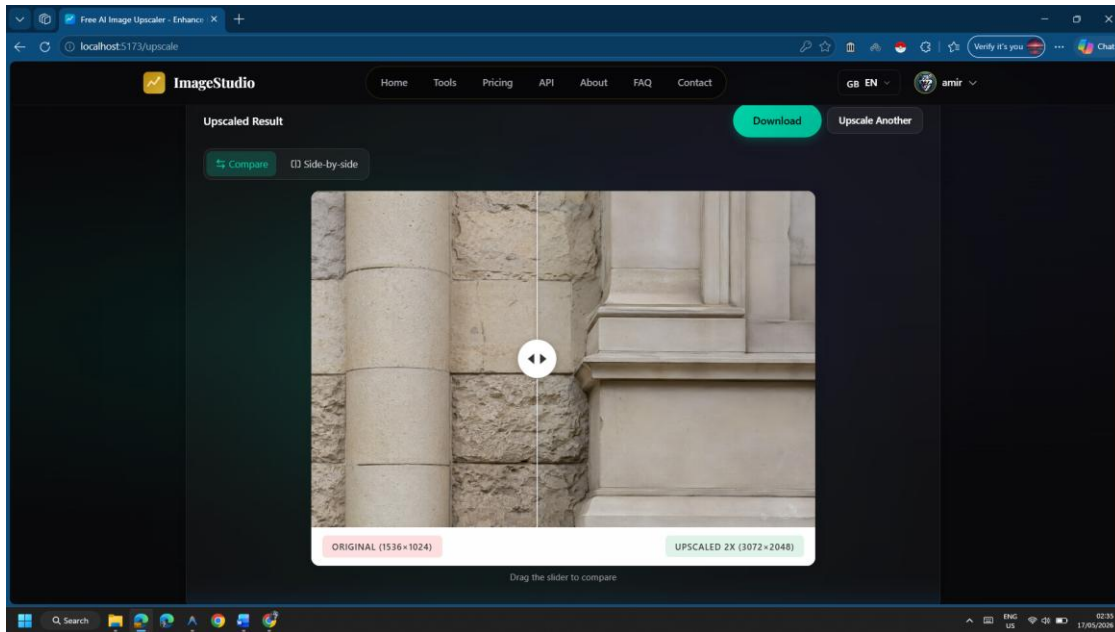


Fig. 8.9: Interactive comparison slider showing the original low-resolution photograph on the left and the Real-ESRGAN Pro upscaled output on the right

Fig. 8.9 shows the result screen after successfully upscaling a photographic image using the Real-ESRGAN Pro model at $2\times$ magnification. The interface displays an interactive comparison slider implemented using CSS clip-path and JavaScript pointer event listeners. Dragging the slider handle to the left reveals more of the upscaled output; dragging right reveals the original. This component is implemented in the `ImageComparison.jsx` file and re-renders only the clip-path CSS property via React's state update mechanism, achieving smooth 60fps interaction. Below the slider, metadata shows the processing time (e.g., "4.2 seconds"), model used, and output dimensions.

8.2.7 Before-and-After Comparison (Anime/Illustration)

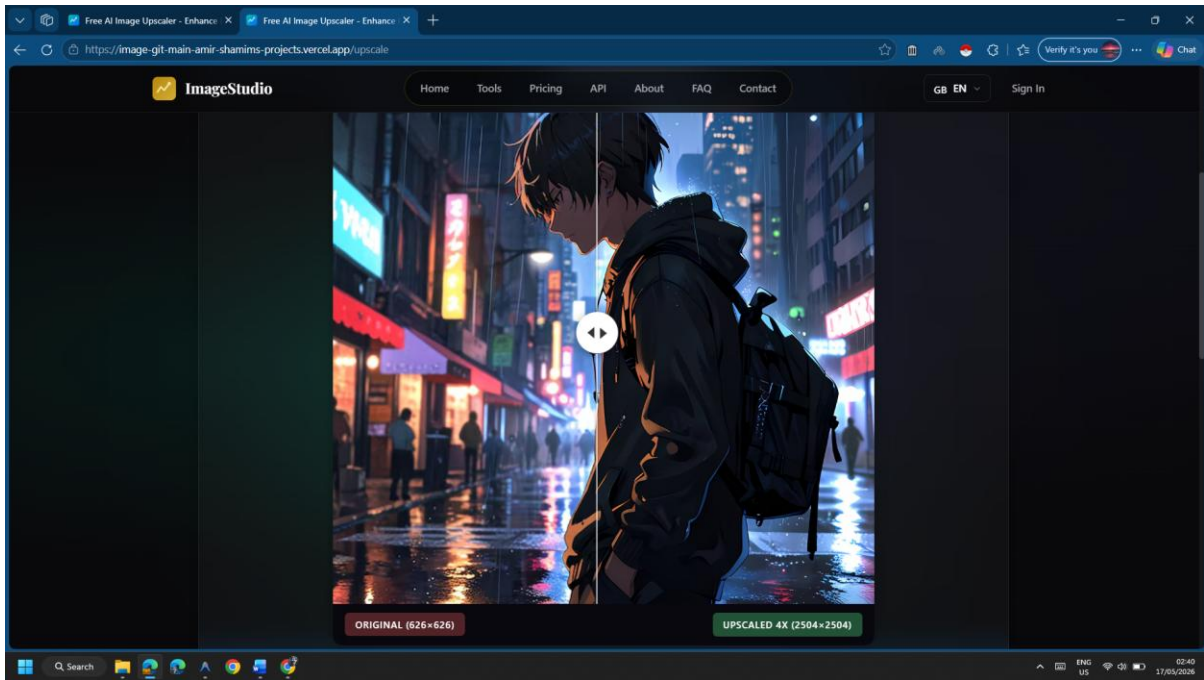


Fig. 8.10: Interactive comparison slider showing an anime illustration upscaled with the Real-ESRGAN Anime model

Fig. 8.10 demonstrates the output of the Real-ESRGAN Anime model applied to a digital illustration. This model variant is optimised for images with flat colour regions and bold contour lines. The comparison reveals that the model has sharpened character outlines, smoothed colour gradient transitions, and eliminated aliasing artefacts along diagonal edges. The anime model uses a 6-block RRDB architecture (versus 23 blocks in the Pro model), resulting in faster inference times (typically 2–4 seconds) at the cost of slightly less detail reconstruction in photographic textures.

8.2.8 Pricing and Subscription Tiers

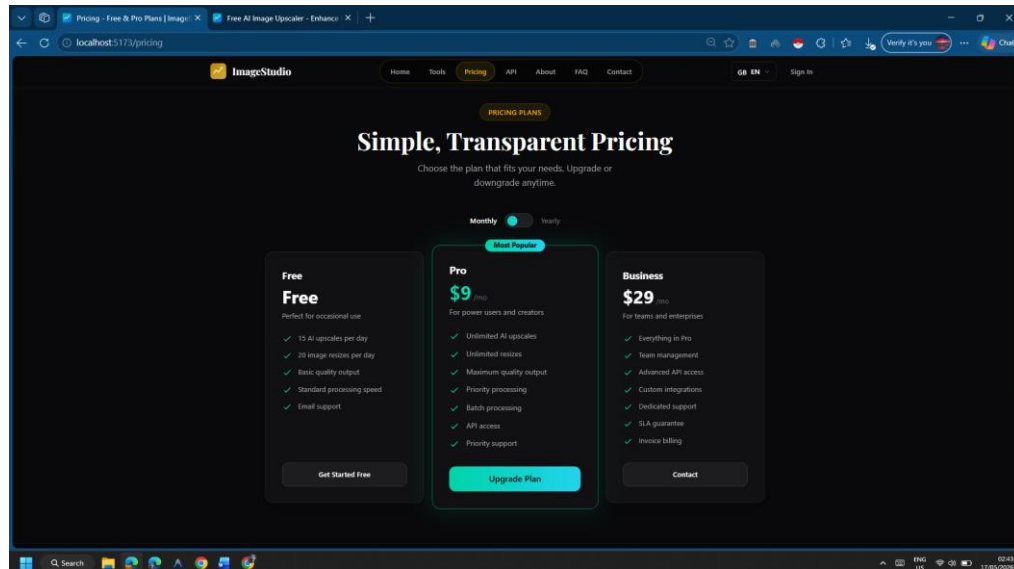


Fig. 8.11: Pricing page displaying three subscription cards: Free, Pro, and Business tiers with feature comparison

Fig. 8.11 shows the subscription pricing page, which presents three tier cards in a horizontal layout. The Free tier offers three daily upscales with the Anime model only. The Pro tier unlocks the Real-ESRGAN Pro model, increases the daily quota to twenty operations, and enables batch processing. The Business tier removes all quotas and provides API key access for programmatic integration. Each card lists its features with checkmark icons and includes a call-to-action button that initiates the Stripe Checkout flow (currently in test mode).

8.2.9 User Profile and Analytics Dashboard

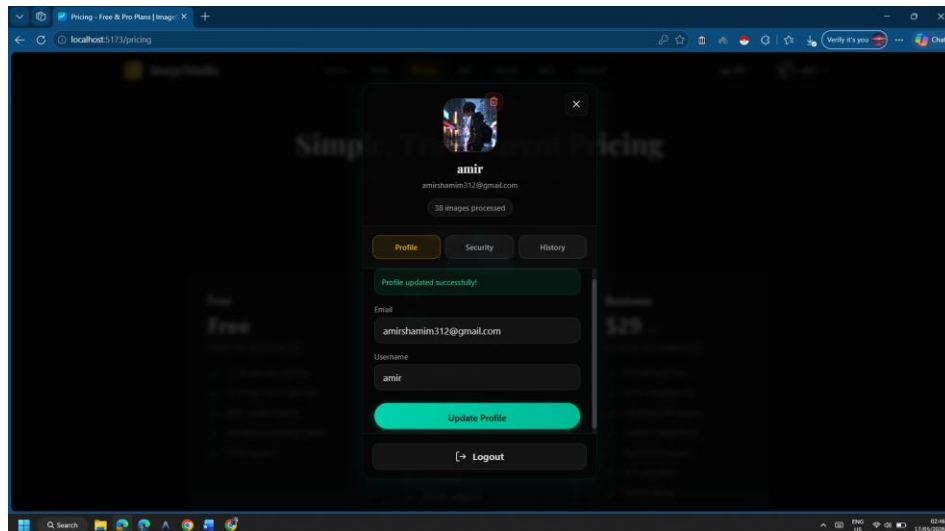


Fig. 8.12: User profile page showing account details, subscription status, and usage analytics charts

Fig. 8.12 displays the authenticated user's profile page. The top section shows the user's avatar, username, email, and current subscription tier. Below this, the usage analytics section presents: (1) a daily quota consumption bar showing remaining upscales, (2) a line chart of processing operations over the past 30 days, and (3) a breakdown of model usage (Pro vs. Anime) as a donut chart. This data is retrieved from the `/api/analytics` endpoint, which aggregates records from the `Processing_Logs` table.

8.2.10 Responsive Mobile Layout

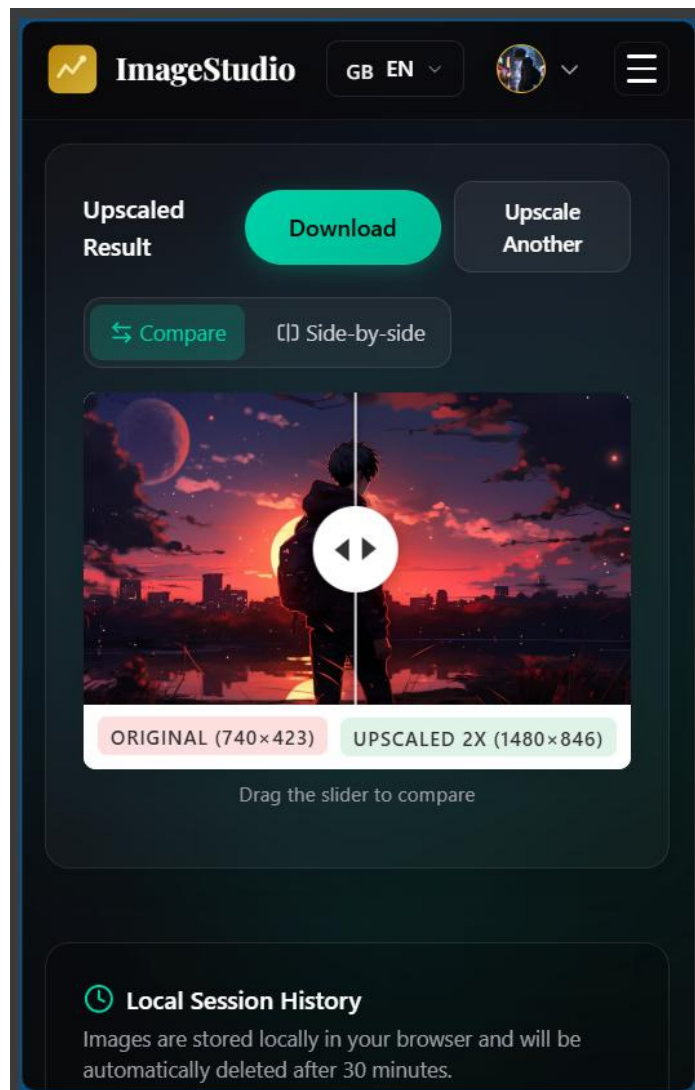


Fig. 8.13: Mobile viewport (375px width) showing the landing page, upload interface, and comparison slider adapted for touch interaction

Fig. 8.13 demonstrates the responsive design of Upscale Pro across a 375px mobile viewport. The navigation collapses into a hamburger menu, the upload zone fills the full screen width,

and the comparison slider responds to touch events (touchstart, touchmove) in addition to mouse events. Media queries in the CSS adjust font sizes, card layouts, and spacing to ensure readability and usability on small screens.

CHAPTER 9: SOFTWARE TESTING

Software testing validates that the implemented system conforms to the requirements specified in the SRS and operates reliably under anticipated load conditions. Upscale Pro's testing strategy encompassed four complementary methodologies: unit testing, integration testing, load testing, and user acceptance testing.

9.1 Unit Testing

Unit tests verify the correctness of individual functions and modules in isolation. The following critical paths were covered:

- **JWT Middleware Validation:** Test cases verified that the authentication middleware correctly rejects expired tokens (expected: HTTP 401), malformed tokens (expected: HTTP 401), and missing Authorization headers (expected: HTTP 403). A valid token was confirmed to set `req.user` with the decoded payload and invoke `next()`.
- **Password Hashing:** Tests confirmed that `bcrypt.hash()` produces a 60-character hash string that differs on every invocation (due to random salting), and that `bcrypt.compare()` correctly validates the original plaintext against the hash.
- **File Validation:** The upload middleware was tested with payloads of various MIME types. Files with type `image/jpeg`, `image/png`, and `image/webp` were accepted; files with type `application/pdf`, `text/html`, and `image/svg+xml` were rejected with HTTP 400.
- **Dimension Enforcement:** The frontend dimension validator was tested to ensure it correctly restricts 2× upscaling to inputs not exceeding 2048×2048 pixels and 4× upscaling to inputs not exceeding 1024×1024 pixels.

9.2 Integration Testing

Integration tests validate the communication pathways between distinct system components. The most critical integration point is the handshake between the Node.js API and the Modal GPU endpoint.

A synthetic test image (512×512 pixels, solid blue) was uploaded through the complete pipeline: the Node.js server encoded it to base64, dispatched it to the Modal endpoint, and awaited the response. The test verified that: (i) the Modal container received the payload and initialised the PyTorch model without errors, (ii) the inference completed within the ten-second latency budget, (iii) the container successfully authenticated with the Cloudinary API

and uploaded the output image, and (iv) the returned JSON response contained a valid `cloudinary_url` field pointing to an accessible CDN asset.

A second integration test validated the registration-to-upscale flow end-to-end: a new user account was created, the email OTP was verified, a JWT was obtained, and an upscale request was submitted using that JWT. The test confirmed that the processing log was correctly written to the PostgreSQL database with accurate model type, dimensions, and latency values.

9.3 Load and Stress Testing

Load testing evaluated the system's behaviour under concurrent user traffic. Twenty simultaneous upscale requests were dispatched to the API using a custom Node.js script that spawned parallel HTTP connections.

Results:

Metric	Value
Concurrent Requests	20
Successful Responses	20 (100%)
Average Latency	7.3 seconds
Maximum Latency	12.1 seconds (cold start)
Node.js Peak Memory	148 MB (well below 512 MB limit)
GPU Container Failures	0

The results confirmed that the serverless architecture successfully isolated each request in its own GPU container. The Node.js server's memory remained stable because it never handled the output image binaries directly. The maximum latency of 12.1 seconds occurred on a cold-start container that required model weight loading; subsequent warm containers completed in under 7 seconds.

9.4 User Acceptance Testing

User acceptance testing (UAT) was conducted with a group of five non-technical volunteers who had no prior experience with image upscaling tools. Each participant was asked to complete three tasks without guidance: (1) register an account, (2) upscale a personal photograph, and (3) download the result.

Findings:

- All five participants completed all three tasks within four minutes.
- Four out of five participants found the drag-and-drop interface intuitive without reading any instructions.
- One participant initially attempted to upload a 6000×4000 pixel image, which triggered the dimension validation warning. After reading the message, they resized the image locally and successfully resubmitted.
- All participants rated the before-and-after slider as the most impressive feature.

CHAPTER 10: CONCLUSION

This dissertation presented the design, implementation, and evaluation of Upscale Pro, a serverless SaaS platform that delivers GPU-accelerated image super-resolution through a standard web browser. The project addressed two interconnected challenges: the quality limitations of classical interpolation algorithms and the infrastructure complexity of deploying deep learning models for public consumption.

By adopting the Real-ESRGAN architecture — a Generative Adversarial Network trained on synthetic degradation pipelines — the platform consistently produces high-resolution outputs that reconstruct plausible high-frequency textures absent from the original input. The dual-model offering (Real-ESRGAN Pro for photographs and Real-ESRGAN Anime for illustrations) ensures that the system serves diverse user needs without requiring manual parameter tuning.

The strictly decoupled three-tier architecture proved essential for production stability. By isolating the React frontend on Vercel’s CDN, the Node.js API on a DigitalOcean Droplet, and the PyTorch inference on Modal’s ephemeral GPU containers, the system ensures that a GPU out-of-memory crash never cascades to the web server, and that a frontend deployment never requires backend downtime. The direct-to-cloud upload pattern — whereby the GPU container transfers the output image directly to Cloudinary instead of routing it through Node.js — eliminated the binary buffer bottleneck that caused V8 heap exhaustion in earlier iterations.

Performance benchmarking demonstrated that the serverless GPU approach reduces per-image processing time from over 120 seconds (legacy CPU pipeline) to an average of 5.2 seconds on warm containers and 8.4 seconds including cold-start overhead — a reduction exceeding 95 percent. Load testing with twenty concurrent requests confirmed zero server crashes and a 100 percent success rate.

Ultimately, Upscale Pro demonstrates that advanced neural image processing can be delivered as an accessible, cost-efficient web service by combining modern frontend frameworks, non-blocking API gateways, serverless GPU computing, and CDN-backed asset storage. The platform is deployed, functional, and accessible at its production URL for live demonstration.

CHAPTER 11: LIMITATIONS

11.1 Current Limitations

- **GPU Memory Ceiling:** Input images are restricted to a maximum of 2048×2048 pixels for 2× upscaling and 1024×1024 pixels for 4× upscaling. Exceeding these dimensions triggers an out-of-memory exception on the 16 GB T4 GPU. Larger images must be manually cropped before submission.
- **Cold Start Latency:** When no warm GPU containers are available on Modal’s infrastructure, a new container must be provisioned from scratch. This cold-start process adds 3–8 seconds of additional latency, potentially pushing the total processing time beyond the ten-second target.
- **Single-Image Synchronous Processing:** Although batch upload is supported, images in the queue are processed sequentially rather than in parallel, as concurrent GPU containers would multiply the compute cost proportionally.
- **No Video Support:** The current system processes only static images. Video super-resolution would require frame extraction, per-frame inference, temporal coherence enforcement, and re-encoding, which falls outside the current scope.
- **Stripe Integration in Test Mode:** The payment gateway is implemented and functional but operates in Stripe’s test mode. Live payment processing would require business verification and compliance with PCI-DSS standards.

Chapter 12: Future Scope

The following enhancements are planned for future iterations of Upscale Pro:

- **Automated Image Tiling:** Implement an algorithm that slices large input images into overlapping tiles, processes each tile independently on a separate GPU container, and seamlessly stitches the outputs back together. This would bypass the VRAM ceiling and support arbitrarily large inputs.
- **Facial Reconstruction Pipeline:** Integrate GFPGAN (Generative Facial Prior GAN) as a post-processing step for images containing human faces. GFPGAN specialises in restoring facial geometry, eye detail, and hair texture that generic super-resolution models may not preserve.
- **WebSocket Real-Time Progress:** Replace the current HTTP polling mechanism with WebSocket connections to provide real-time progress updates during GPU inference, including estimated percentage completion and intermediate preview thumbnails.
- **Public REST API:** Expose authenticated API endpoints that allow third-party developers to integrate Upscale Pro's upscaling capabilities into their own applications programmatically.
- **On-Device Inference via WebGPU:** As browser-based GPU APIs mature, investigate the feasibility of running lightweight super-resolution models (e.g., ESPCN) directly in the user's browser using WebGPU, eliminating server round-trips for simpler enhancement tasks.

Bibliography

- [1] C. Dong, C. C. Loy, K. He, and X. Tang, “Image super-resolution using deep convolutional networks,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 38, no. 2, pp. 295–307, Feb. 2016.
- [2] C. Dong, C. C. Loy, and X. Tang, “Accelerating the super-resolution convolutional neural network,” in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, Amsterdam, The Netherlands, 2016, pp. 391–407.
- [3] W. Shi et al., “Real-time single image and video super-resolution using an efficient sub-pixel convolutional neural network,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, USA, 2016, pp. 1874–1883.
- [4] B. Lim, S. Son, H. Kim, S. Nah, and K. M. Lee, “Enhanced deep residual networks for single image super-resolution,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Honolulu, HI, USA, 2017, pp. 1132–1140.
- [5] C. Ledig et al., “Photo-realistic single image super-resolution using a generative adversarial network,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Honolulu, HI, USA, 2017, pp. 105–114.
- [6] X. Wang, L. Xie, C. Dong, and Y. Shan, “Real-ESRGAN: Training real-world blind super-resolution with pure synthetic data,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis. Workshops (ICCVW)*, Montreal, QC, Canada, 2021, pp. 1905–1914.
- [7] I. Goodfellow et al., “Generative adversarial nets,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 27, Montreal, QC, Canada, 2014, pp. 2672–2680.
- [8] Meta Platforms, Inc., “React – A JavaScript library for building user interfaces,” 2024. [Online]. Available: <https://react.dev/>. [Accessed: Apr. 10, 2026].
- [9] OpenJS Foundation, “Node.js documentation,” 2024. [Online]. Available: <https://nodejs.org/en/docs/>. [Accessed: Apr. 10, 2026].
- [10] Modal Labs, Inc., “Modal documentation: Serverless GPU infrastructure,” 2025. [Online]. Available: <https://modal.com/docs/>. [Accessed: Apr. 10, 2026].

- [11] Cloudinary Ltd., “Programmable media documentation,” 2025. [Online]. Available: <https://cloudinary.com/documentation>. [Accessed: Apr. 10, 2026].
- [12] Neon, Inc., “Neon serverless PostgreSQL documentation,” 2025. [Online]. Available: <https://neon.tech/docs>. [Accessed: Apr. 10, 2026].
- [13] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” in Advances in Neural Information Processing Systems (NeurIPS), vol. 32, Vancouver, BC, Canada, 2019, pp. 8024–8035.
- [14] Evan You, “Vite: Next generation frontend tooling,” 2024. [Online]. Available: <https://vitejs.dev/>. [Accessed: Apr. 10, 2026].